



ÉCOLE NATIONAL SUPÉRIEUR D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES

FILIÈRE INGÉNIERIE DES SYSTÈMES EMBARQUÉS ET MOBILES

Mémoire de projet de fin de 2ème année

SÉCURITÉ DES VÉHICULES ET ASSISTANCE À LA CONDUITE SOUS RASPBERRY PI

Soutenu par :
ELHARI Chadi
LAHLAL Hamza

Encadré par :
MR. RAHAL Romadi

Résumé

Le présent travail de PFA tache de créer un système de sécurité et d'assistance à la conduite pour les véhicules automobiles. Les deux sous-systèmes sont conçus et développés sur un nano-ordinateur monocarte Raspberry Pi 4B, pour tirer avantage des dernières améliorations disponibles sur le marché tant sur le niveau matériel que logiciel. L'élaboration de ces deux systèmes fut sujet de l'utilisation, en plus du nano-ordinateur Raspberry Pi, de capteurs au niveau matériel, à savoir : Un module caméra (Picamera à couleurs classiques basée sur un capteur de type CMOS 8MP) et un module GPS (Module GPS ublox / u-blox NEO6M avec antenne et EEPROM intégrée). Au niveau logiciel, les bibliothèques de traitement d'images en temps réel, de reconnaissance faciale et de détection d'objet : OpenCV Tensorflow et Keras ont été utilisés. Les deux modules développés sont les suivants :

- Module de sécurité : Utilisation de la reconnaissance faciale et du suivi par GPS pour la sécurité du véhicule.
- Module assistance de conduite : Utilisation de la détection des panneaux de circulation, et des données GPS pour permettre au conducteur, grâce à une interface, d'accéder aux données permettant d'assister à sa conduite tel que : La vitesse maximale autorisée dans la zone et la détection et mémorisation des panneaux de circulation.

Notre travail a été réalisé selon plusieurs phases à savoir : la phase d'étude des composants matériels nécessaires à la réalisation du projet. La phase du développement dans laquelle nous avons mobilisé les outils d'apprentissage automatique et de programmation embarquée. Et en dernier lieu une phase de rédaction et réalisation de ce rapport. Le rapport est divisé en 3 parties principales :

- I Apprentissage automatique et détection.
- II Analyse et Conception.
- III Réalisation.

Abstract

The 21st century is arguably the golden age of automation. Where machine learning algorithms dwell on every device, vehicle automation has been glowing with advances in object detection algorithms and hardware conception and utilization. The exponential shrinking in transistor size, made it possible today to create and develop machine-learning systems that can be embedded on vehicles to improve the quality of driving, but also the security of the vehicle in question. Autonomous cars create and maintain a map of their surroundings based on a variety of sensors situated in different parts of the vehicle. Radar sensors monitor the position of nearby vehicles. Video cameras detect traffic lights, read road signs, track other vehicles, and look for pedestrians. This project harnesses the powerfull capabilities of machine learning and mobilizes the flexible uses of powerfull nano-computers to unveil new horizons in the field of vehicle automation.

Introduction

Les voitures autonomes ne relèvent désormais plus du monde de science-fiction. Pendant la réalisation de cette mémoire, des constructeurs tels que Tesla, BMW et Mercedes sont déjà penchés sur la commercialisation des véhicules avec plusieurs systèmes de contrôle autonomes. En effet l'automatisation des voitures a pris deux principales voies :

Les systèmes de contrôle offrant une certaine autonomie au véhicule et ne nécessitant aucune intervention humaine. Ces systèmes sont généralement utilisés en guise de sécurité et pour protéger l'utilisateur du véhicule. Ces systèmes ont tendance à traiter les données en temps réel pour aboutir à des sorties concises et des fois pour des situations critiques. Un exemple est le système de sécurité par reconnaissance facial et suivi par GPS qu'on construit au long de ce projet, mais aussi le système de détection d'obstacles suivi d'un freinage, ou le système de contrôle de trajectoire à très faible allure sans intervention du conducteur (dans les embouteillages par exemple) : Deux systèmes présents aujourd'hui sur plusieurs véhicules automobiles.

Le deuxième type concerne les systèmes d'aide à la conduite, qui visent en premier lieu à assister le conducteur dans ses actions afin d'optimiser le contrôle du véhicule ou la conduite en générale. Une instance de ce système serait le système de détection et mémorisation des panneaux de circulation, et le système de détermination de la vitesse maximale par données GPS, qu'on développe au cours de ce projet. On peut également citer le système de régularisation de la pression du freinage, déclenché en cas de passage anormalement rapide entre les pédales.

Table des matières

1	Apprentissage automatique et détection	1
1.1	Machine Learning	1
1.2	Types d'apprentissage	3
1.2.1	Apprentissage supervisé	3
1.2.2	Apprentissage non-supervisé	4
1.2.3	Apprentissage par renforcement	5
2	Analyse et Conception	6
2.1	Composants matériels : HARDWARE	6
2.1.1	Raspberry Pi	6
2.1.2	Module caméra : Picamera	8
2.1.3	Module GPS : Neo 6M V2 GPS	9
2.2	Environnement logiciel : SOFTWARE	10
2.2.1	Python	10
2.2.2	OpenCV	11
2.2.3	TensorFlow	12
2.2.4	Keras	12
2.2.5	Numpy et Scikit Image	13
2.2.6	SciPy	13
2.2.7	Dlib et FaceRecognition	14
2.2.8	OverPass	14
2.2.9	Pillow	15
2.3	Systèmes de sécurité et assistance à la conduite	16
2.3.1	Système de sécurité par reconnaissance faciale et suivi par GPS	16
2.3.2	Système d'assistance à la conduite	17
2.3.3	Technologies utilisées	18
3	Réalisation	20
3.1	Circuit utilisé	20
3.1.1	Configuration du module GPS	21
3.2	Installation des bibliothèques et dépendances	22
3.3	Système de détection et reconnaissance faciale	26
3.4	Système de détection des panneaux, et vitesse maximale.	32
3.4.1	Détection des panneaux de circulation	32
3.4.2	Détermination de la vitesse maximale	38
3.5	Résultats et futur du projet	39
3.5.1	Performance	39
3.5.2	Problèmes au niveau matériel	40

3.5.3	Influence de l'environnement du système	41
4	Conclusion	42
	Bibliographie	43

Table des figures

1.1	Programmation traditionnelle vs Machine learning.[1]	1
1.2	Étapes de l'apprentissage automatique	2
1.3	Types d'apprentissage automatique[1]	3
1.4	Procédure de l'apprentissage supervisé	3
1.5	Procédure de l'apprentissage non-supervisé	4
1.6	Procédure de l'apprentissage par renforcement	5
2.1	Accès distant avec VNC Viewer, et accès par SSH via Putty.	6
2.2	Raspbeery Pi 4B	6
2.3	Système d'exploitation Raspbian	8
2.4	Module caméra : Picamera	8
2.5	Positions d'installation du Picamera.	9
2.6	Module GPS : Neo 6M V2 GPS	9
2.7	Python et Machine Learning	10
2.8	Logo OpenCV	11
2.9	Seuillage sous OpenCV	11
2.10	TensorFlow	12
2.11	Keras	12
2.12	Numpy et Scikit Image.	13
2.13	SciPy	13
2.14	Dlib	14
2.15	OverPass	14
2.16	Pillow	15
2.17	Procédure de déverrouillage par reconnaissance faciale	16
2.18	Mécanisme de sécurité	16
2.19	Détection et mémorisation des panneaux de circulation	17
2.20	Détection de la vitesse maximale	17
2.21	Déscripteurs de HAAR	18
2.22	Utilisation des descripteurs de HAAR	18
2.23	Histogram of Oriented Gradients HOG	19
2.24	Panneaux de circulation récupérés après filtrage par contour et couleur.	19
3.1	Branchement des modules.	20
3.2	Circuit complet	20
3.3	Lancement du module GPS	21
3.4	First Fix	21
3.5	Installation des dépendances OpenCV	22
3.6	Installation des dépendances OpenCV	22

3.7	Téléchargement des sources OpenCV	23
3.8	Compilation des sources OpenCV	23
3.9	Compilation des sources OpenCV	24
3.10	Test de vérification OpenCV	24
3.11	Installation des dépendances TensorFlow et Keras	25
3.12	Installation des dépendances TensorFlow et Keras	25
3.13	Execution du système de reconnaissance faciale	26
3.14	Demarrer l'ajout d'une nouvelle personne au système de reconnaissance.	26
3.15	Renseignement des information sur la personne à ajouter au système.	27
3.16	Enregistrement du visage du sujet à travers la détection faciale.	27
3.17	Enregistrement des nouveaux visages.	28
3.18	Codage et sérialisation des nouvelles données.	28
3.19	Démarrage du système de reconnaissance faciale.	29
3.20	Reconnaissance faciale.	29
3.21	Reconnaissance faciale.	30
3.22	Visage non reconnu détecté.	30
3.23	Notification d'intrusion.	31
3.24	Détection du panneau 'STOP'.	32
3.25	Détection du panneau de limitation de vitesse.	33
3.26	Détection du panneau de limitation de vitesse.	33
3.27	Détection du panneau d'interdiction de doublage.	34
3.28	Détection d'un panneau d'interdiction.	34
3.29	Détection d'un panneau d'obligation.	35
3.30	Détection d'un panneau d'obligation.	35
3.31	Détection du panneau d'interdiction de stationnement.	36
3.32	Détection du panneau de danger : Succession de virages'.	36
3.33	Détection du panneau : 'Barrière de neige'.	37
3.34	Détection du panneau : 'Cédez le passage'.	37
3.35	Fonctionnement du système d'assistance à la conduite	38
3.36	Neural Compute Stick 2 par Intel	39
3.37	Module GPS grillé	40

1 Apprentissage automatique et détection

Vous avez sans doute entendu parler de l'intelligence artificielle, l'apprentissage automatique ou l'apprentissage approfondi, En général et depuis 2019 les domaines d'études révolutionnant autour de l'intelligence artificielle ont connu une explosion inédite et d'ailleurs incontournable en raison du rétrécissement exponentiel de la taille du transistor permettant encore plus de calculs.

1.1 Machine Learning

L'apprentissage automatique (en anglais Machine Learning, est un champ d'étude de l'intelligence artificielle qui se fonde sur des approches mathématiques et statistiques pour permettre aux machines (Ordinateurs) d'apprendre à partir des données, En d'autres termes, c'est donner aux ordinateurs la capacité d'améliorer leurs performances à résoudre des tâches sans être explicitement programmés pour chacune.[1]

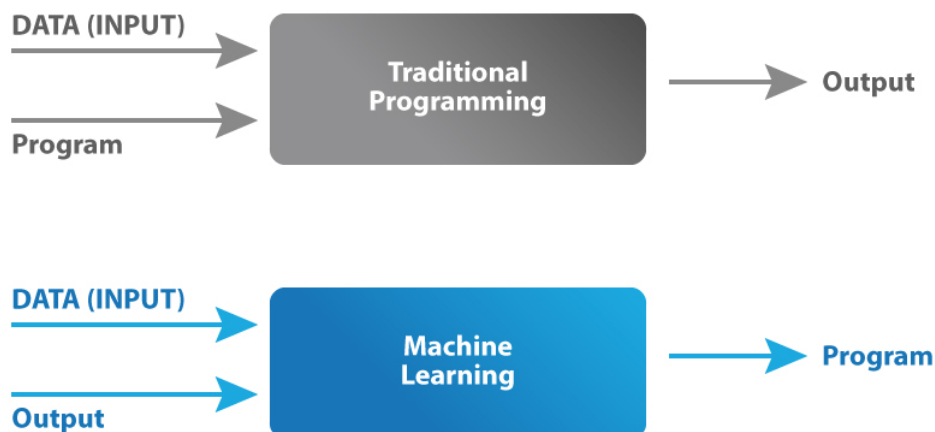


FIGURE 1.1 – Programmation traditionnelle vs Machine learning.[1]

L'objectif de la recherche en intelligence artificielle est de doter un système informatique de capacités de réflexion similaires à celles des humains. Il y a donc un défi dans la compréhension du raisonnement humain, mais surtout dans la modélisation et dans la reproduction de celui-ci.

Le processus commence en premier temps par alimenter des données de bonne qualité, à la machine pour l'entraîner, ceci est fait en construisant des modèles d'apprentissage automatique à travers les données et les différents algorithmes.

Le choix des algorithmes dépend du type de données à traiter mais également de la tâche que nous essayons d'automatiser.

L'apprentissage automatique s'applique à un grand nombre d'activités humaines et convient en particulier au problème de la prise de décision automatisée. Il s'agira, par exemple :

- De déclencher un processus d'alerte en fonction de signaux reçus par des capteurs ;
- De la reconnaissance des formes ;
- De contrôler un processus et de diagnostiquer des pannes ;

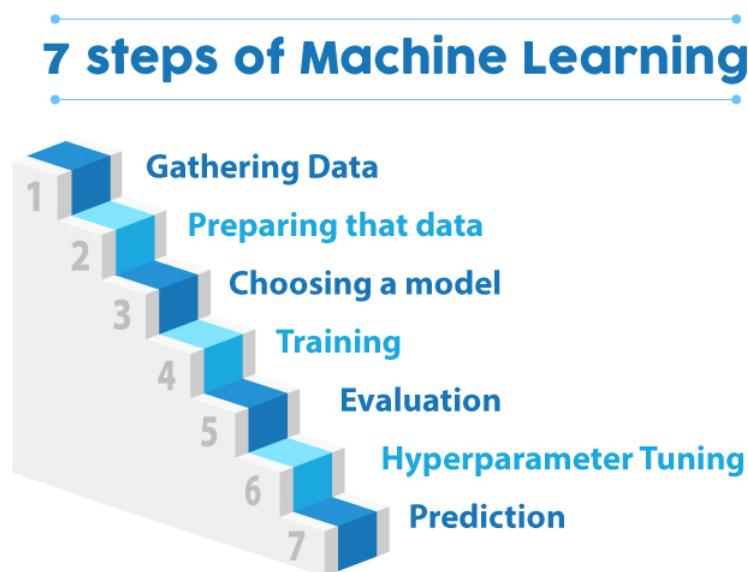


FIGURE 1.2 – Étapes de l'apprentissage automatique

1.2 Types d'apprentissage

Les algorithmes d'apprentissage peuvent se catégoriser selon le mode d'apprentissage qu'ils emploient :

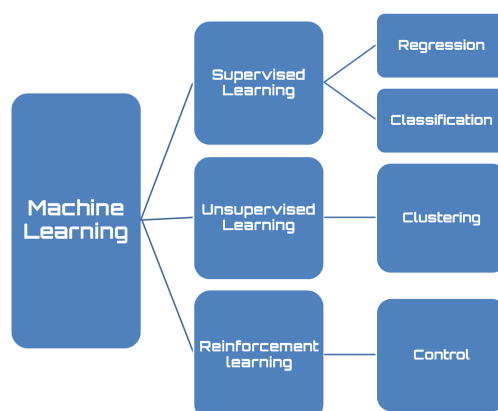


FIGURE 1.3 – Types d'apprentissage automatique[1]

1.2.1 Apprentissage supervisé

Dans le cas où les classes sont prédéterminées et les exemples connus, le système apprend à classer selon un modèle de classement ; on parle alors d'apprentissage supervisé (ou d'analyse discriminante). Un expert (ou oracle) doit préalablement correctement étiqueter des exemples pour construire un Dataset. L'apprenant peut alors trouver ou approximer la fonction qui permet d'affecter la bonne « étiquette » à ces exemples. Parfois il est préférable d'associer une donnée non pas à une classe unique, mais une probabilité d'appartenance à chacune des classes prédéterminées (on parle alors d'apprentissage supervisé probabiliste).

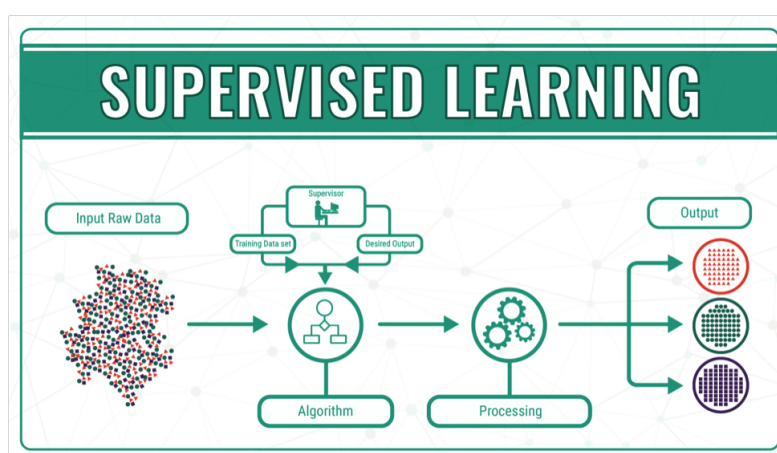


FIGURE 1.4 – Procédure de l'apprentissage supervisé

L'analyse discriminante linéaire ou les SVM sont des exemples typiques et propres à ce type d'apprentissage.

1.2.2 Apprentissage non-supervisé

Dans ce type d'apprentissage le système ou l'opérateur ne dispose que d'exemples, et non d'étiquettes, le nombre de classes et leur nature n'ont pas été prédéterminés. Aucun expert n'est disponible ni requis. L'algorithme doit découvrir par lui-même la structure plus ou moins cachée des données.

Le système doit ici dans l'espace de description cibler les données selon leurs attributs disponibles, pour les classer en groupe homogènes d'exemples. (Clustering) La similarité est généralement calculée selon la fonction de distance entre les paires d'exemples.

C'est ensuite à l'opérateur d'associer ou déduire un sens pour chaque groupement. Divers outils mathématiques et logiciels peuvent l'aider. On parle également d'analyse des données en régression. Si l'approche est probabiliste (c'est à dire que chaque exemple au lieu d'être classé dans une seule classe est associé aux probabilités d'appartenir à chacune des classes), on parle alors de « Soft Clustering » (par opposition au « Hard Clustering »).

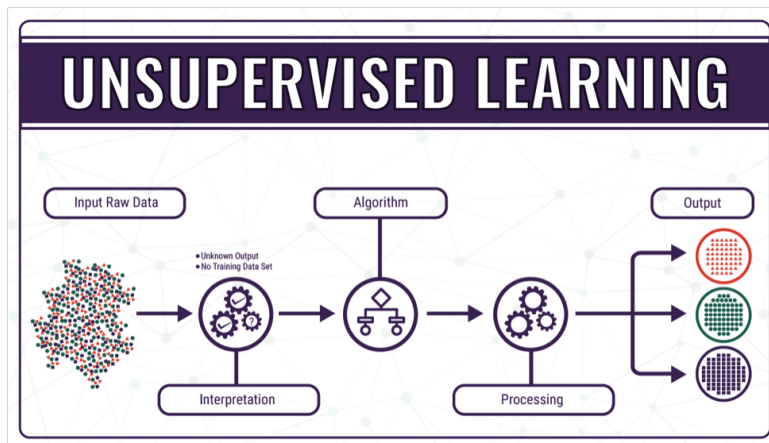


FIGURE 1.5 – Procédure de l'apprentissage non-supervisé

Exemple : Un épidémiologiste pourrait par exemple dans un ensemble assez large de victimes du cancer de foie tenter de faire émerger des hypothèses explicatives, l'ordinateur pourrait différencier différents groupes, qu'on pourrait ensuite associer par exemple à leur provenance géographique, génétique, à l'alcoolisme ou à l'exposition à un métal lourd ou à une toxine telle que l'aflatoxine.

Parmi les algorithmes d'apprentissage non supervisé les plus notables :

- K-MEANS
- Classification hiérarchique
- Le clustering par décalage moyen
- Apriori

1.2.3 Apprentissage par renforcement

Cette fois l'algorithme apprend un comportement étant après une observation. L'action de l'algorithme sur l'environnement produit une valeur de retour qui guide l'algorithme.

Plongé dans un environnement donné, la procédure apprend en recevant des récompenses ou des pénalités en fonction de ses actions. Au travers de son expérience, l'agent cherche à trouver la stratégie décisionnelle optimale capable de lui permettre de maximiser les récompenses accumulées au cours du temps.

De manière générale, l'apprentissage automatique correspond à l'apprentissage réalisé par des algorithmes à partir de données en vue de réaliser des prédictions ou de prendre des décisions. En ce sens, l'apprentissage par renforcement représente l'un des trois paradigmes d'apprentissage automatique aux côtés de l'apprentissage supervisé et l'apprentissage non-supervisé.

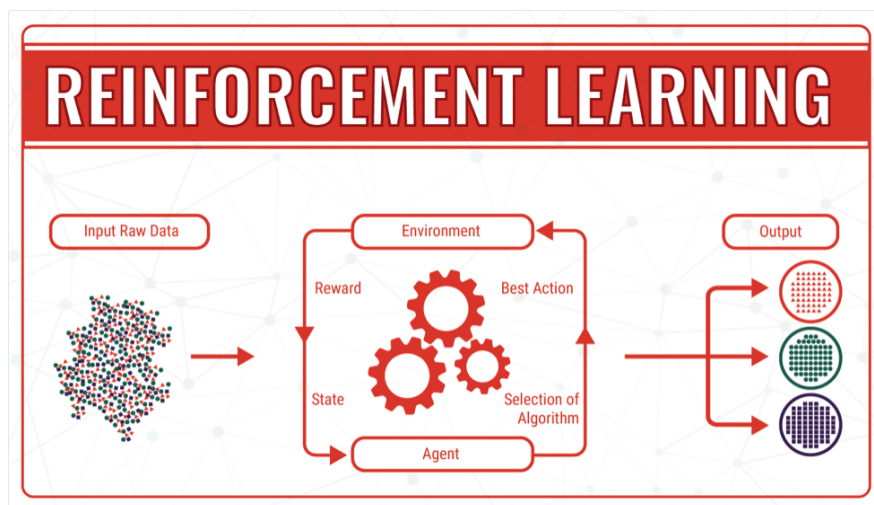


FIGURE 1.6 – Procédure de l'apprentissage par renforcement

Parmi les algorithmes d'apprentissage par renforcement les plus notables, on peut citer :

- Q-Learning
- Model based alrothims

2 Analyse et Conception

2.1 Composants matériels : HARDWARE

La réalisation de ce projet a nécessité l'élaboration de certains choix au niveau matériel afin de satisfaire les besoins de performance, de mobilité et en termes de coût.

En effet le développement des deux systèmes fut entièrement réalisé sur un nano-ordinateur monocarte Raspberry Pi 4B, muni d'un système d'exploitation Raspbian, mais également grâce à deux modules d'interface : Un module caméra et un module GPS.

L'ordinateur Raspberry Pi est fourni nu, c'est-à-dire la carte mère seule, sans boîtier ni écran, d'où la nécessité d'une autre machine pour programmer l'ordinateur Raspberry, par voie SSH ou à travers un accès distant en utilisant les logiciels comme VNC Viewer.

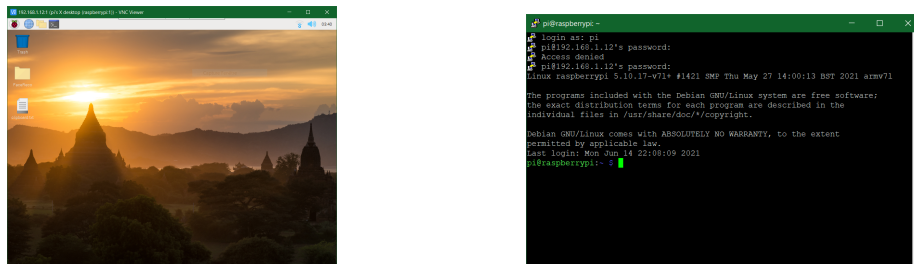


FIGURE 2.1 – Accès distant avec VNC Viewer, et accès par SSH via Putty.

2.1.1 Raspberry Pi

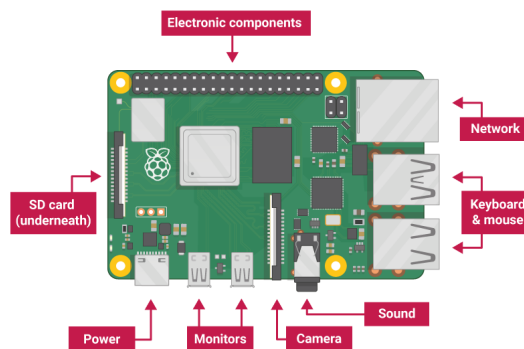


FIGURE 2.2 – Raspbeery Pi 4B

La conception de plus petits composants a permis pendant les années 70, la production d'ordinateurs à des prix accessibles. Tous les composants électroniques nécessaires au fonctionnement de ces ordinateurs tenaient sur un seul support : Un circuit imprimé. Ceci fut la naissance des ordinateurs monocartes. Cette même révolution, donne naissance aujourd'hui au Raspberry Pi 4B : Un nano-ordinateur, monocarte doté d'un système sur puce (SoC).

La dernière itération en date du Raspberry Pi, la version 4B, offre des performances plus élevées que les modèles précédents, puisque le SoC garantit lui-même plus de puissance que la mouture précédente (3B+) et supporte même les vidéos encodées en 4K (HEVC).

Raspberry pi 4 dispose d'un processeur BCM2711 Quad Core hébergeant des cœurs ARM Cortex-A72 cadencés à 1,5 GHz chacun, d'un processeur graphique VideoCore VI cadencé à 500 MHz, d'une mémoire vive (RAM) de 2 Go, d'une radio permettant la communication sans fil (WiFi 5.0 et Bluetooth 5.0) avec les normes IEEE 802.11b/g/n/ac (2,4 et 5 GHz) et de plusieurs ports (2x micro-HDMI, 2x USB 2.0, 2x USB 3.0, Ethernet Gigabit LAN, CSI, DSI)[2]

Raspberry Pi peut servir à des utilisations diverses et dans plusieurs domaines. Parmi les projets notables réalisés grâce à ce dernier :

- Pi-hole, agissant comme un bloqueur de publicité au niveau du réseau.
- PirateBox, permettant aux utilisateurs qui y sont connectés d'échanger des fichiers anonymement et de manière locale.
- RetroPie, Recalbox et Batocera.linux, trois systèmes d'exploitation spécialisés dans le rétrogaming faisant du Raspberry Pi une véritable console de jeu rétrogaming.

Raspbian

Raspbian [2] est un système d'exploitation libre basé sur la distribution GNU/Linux Debian, et optimisé pour le plus petit ordinateur du monde : Raspberry Pi.

La Raspberry Pi est une framboise merveilleuse, mais elle reste dotée d'une puissance inférieure à celle d'un ordinateur moderne. Il est donc préférable d'utiliser un système optimisé pour elle, ce qui est le cas de Raspbian.

Finalement, Raspbian est sans doute la distribution la plus utilisée pour les Raspberry, et bénéficie donc d'une communauté large et active.

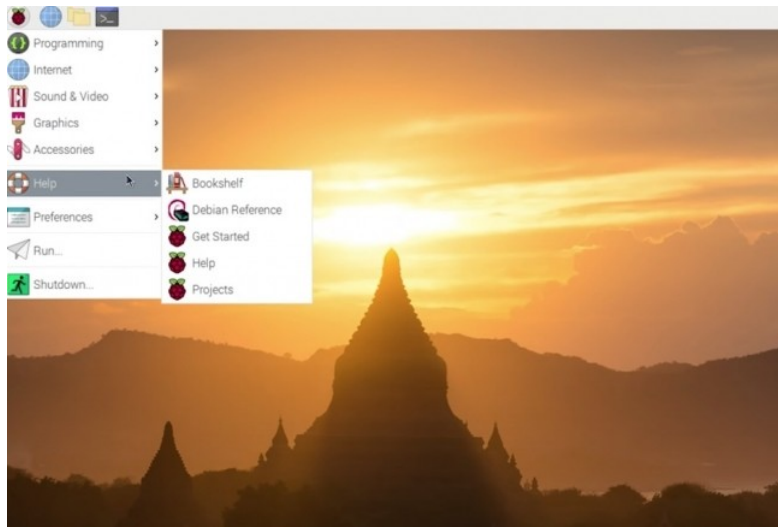


FIGURE 2.3 – Système d'exploitation Raspbian

2.1.2 Module caméra : Picamera

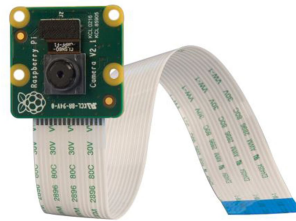


FIGURE 2.4 – Module caméra : Picamera

Le Raspberry Pi Camera Module v2 est une carte complémentaire conçue sur mesure pour Raspberry Pi, avec un objectif fixe. Il est capable de capturer des images statiques 3280 x 2464 pixels, et prend également en charge les vidéos 1080p30, 720p60 et 640x480p90. Il se fixe à Pi par l'une des petites prises de la surface supérieure de la carte et utilise l'interface CSI dédiée, conçue spécialement pour l'interface avec les caméras.

Caractéristiques :

- Taille : 25x24x9
- Résolution : 8 Megapixels
- Pilote : V4L2 (Linux)
- Capteur ; Sony IMX219
- Taille du pixel : 1.12 μm x 1.12 μm
- Distance focale : 3.04 mm

Picamera serait mobilisée dans ce projet comme capteur d'images, et serait installée dans deux positions différentes, plus particulièrement :

- En position d'accès au siège du conducteur
- En position de Dashcam pour réaliser un suivi de la circulation



FIGURE 2.5 – Positions d'installation du Picamera.

2.1.3 Module GPS : Neo 6M V2 GPS

Neo 6M V2 GPS est un module GPS simple doté d'une antenne, d'un adaptateur, d'une batterie et d'un port d'extension d'antenne sur une carte de circuit imprimé. Les modules de type GY-GPS6MV2 sont équipés d'un GPS Ublox NEO-6M. Ils communiquent via liaison série à 9600 bauds par défaut (broches Rx, Tx). Ils sont équipés d'un régulateur, ce qui permet de les alimenter en 5V.



FIGURE 2.6 – Module GPS : Neo 6M V2 GPS

Caractéristiques :

- Type de recepteur : 50 channels, GPS L1(1575.42Mhz)
- Précision du positionnement horizontal : 2.5 m
- Fréquence d'actualisation : 1HZ (5Hz maximum)
- Sensibilité de la navigation ; -161dBm
- Protocoles de communication : NMEA, UBX Binary, RTCM
- Baud rate : 4800-230400 (9600 par défaut)

2.2 Environnement logiciel : SOFTWARE

Afin de développer les deux systèmes en question, il fallait construire un environnement de développement logiciel permettant de satisfaire les différents besoins et fonctionnalités du système en sélectionnant les langages et bibliothèques convenables.

2.2.1 Python



FIGURE 2.7 – Python et Machine Learning

Python[3] est une plateforme complète et généraliste pour le développement logiciel, très facile d'accès et capable de se spécialiser de manière très pointue dans la majorité des domaines informatiques. Python est utilisé par un public très large d'utilisateurs : Des développeurs Web professionnels, des chercheurs en intelligence artificielle ou en bio-informatique, des administrateurs systèmes, ou même des programmeurs occasionnels. C'est le mélange de polyvalence et de facilité qui fait la force de Python.

Le développement de Python ayant commencé à la fin des années 1980, son déploiement dans l'industrie a commencé vers les années 2000. Aujourd'hui, Python est devenu très populaire auprès des développeurs : beaucoup de projets viennent peupler un écosystème déjà très riche, et ce dans tous les domaines. La plateforme bénéficie donc d'une visibilité croissante, qui s'accroîtra encore plus dans les prochaines années.

Python est l'un des langages les plus utilisés en intelligence artificielle. Il dispose d'un riche écosystème et de nombreuses bibliothèques de Deep Learning et Machine Learning disposent d'une interface python.

2.2.2 OpenCV



FIGURE 2.8 – Logo OpenCV

OpenCV (Open Computer Vision)[4] est une bibliothèque graphique spécialisée dans le traitement d'images, que ce soit pour de la photo ou de la vidéo. Initialement développée par Intel sa première version sort en juin 2000, elle est disponible sur la plupart des systèmes d'exploitation et existe pour les langages Python, C++ et Java.

OpenCV fournit un ensemble de plus de 2500 algorithmes de vision par ordinateur, accessibles au travers d'API. Ce qui permet d'effectuer tout un tas de traitements sur des images (extraction de couleurs, détection de visages, de formes, application de filtres,...). Ces algorithmes se basent principalement sur des calculs mathématiques, concernant surtout les traitements sur les matrices.

Les algorithmes d'OpenCV permettent d'appliquer divers traitements sur les images pour faciliter la détection d'éléments précis dans celles-ci, à savoir :

- Le Seuillage
- Les filtres de détection de contours
- Les filtres de lissage
- Les filtres morphologiques

Certains algorithmes d'OpenCV utilisent de l'intelligence artificielle pour faciliter le traitement des images. Dans les faits, cette IA sert à améliorer les algorithmes précédemment détaillés pour obtenir des résultats plus précis.

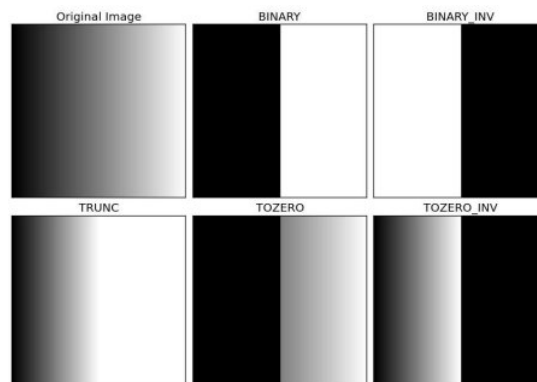


FIGURE 2.9 – Seuillage sous OpenCV

2.2.3 TensorFlow



FIGURE 2.10 – TensorFlow

L'équipe Google Brain a créé TensorFlow en 2011, pour être un système propriétaire dédié aux réseaux de neurones de Deep Learning, TensorFlow s'appelait à l'origine DistBelief. Par la suite, le code source de DistBelief a été modifié et cet outil est devenu une interface de programmation d'application. En 2015, il a été renommé TensorFlow et Google l'a rendu open source. Il a subi plus de 21000 modifications par la communauté et est passé en version 1.0 en février 2017.

TensorFlow [5] regroupe un grand nombre de modèles et d'algorithmes de Machine Learning et de Deep Learning. Son API front-end de développement d'applications repose sur le langage de programmation Python, tandis que l'exécution de ces applications s'effectue en C++.

2.2.4 Keras



FIGURE 2.11 – Keras

Keras est une bibliothèque open source écrite en Python (sous licence MIT) basée principalement sur les travaux du développeur de Google François Chollet dans le cadre du projet ONEIROS (Open-ended Neuro-Electronic Intelligent Robot Operating System). Une première version du logiciel multiplateforme a été publiée le 28 mars 2015.

Le but de cette bibliothèque est de permettre la constitution rapide de réseaux neuronaux. Dans ce cadre, Keras ne fonctionne pas comme un framework propre mais comme une interface de programmation applicative (API) pour l'accès et la programmation de différents frameworks d'apprentissage automatique. Theano, Microsoft Cognitive Toolkit (anciennement CNTK) et TensorFlow font notamment partie des frameworks pris en charge par Keras.

Depuis la sortie de TensorFlow 1.4, Keras fait partie intégrante de l'API de base de TensorFlow.

Toutefois, la bibliothèque est toujours développée comme un logiciel indépendant, car l'approche initiale visant à l'utiliser comme une interface pour différents frameworks est toujours d'actualité.

Keras est une bibliothèque agissant au niveau du modèle : elle met à disposition des modules permettant de développer des modèles de deep learning (apprentissage profond) complexes. Contrairement aux frameworks indépendants, ce logiciel open source ne s'occupe pas des opérations « low level » et utilise à cet effet les bibliothèques de frameworks d'apprentissage automatique associés qui tiennent pratiquement lieu de moteurs back-end pour Keras. Conformément au principe de modularité, les couches désirées pour le réseau neuronal à mettre en place sont connectées. Pour autant, il n'est pas nécessaire de comprendre l'infrastructure réelle du framework choisi et l'utilisateur de Keras n'a pas à la démarrer directement.

2.2.5 Numpy et Scikit Image



FIGURE 2.12 – Numpy et Scikit Image.

Numpy est l'une des bibliothèques principales de la programmation Python et prend en charge les tableaux. Une image est représentée par un tableau Numpy standard contenant des pixels : Les plus petits blocs de données formant une image. Par conséquent, en utilisant les opérations NumPy de base, telles que le découpage en tranches (slicing), les masques et toute sorte d'indexation, nous pouvons modifier les valeurs de pixel d'une image. L'image peut être chargée à l'aide de skimage et affichée à l'aide de matplotlib.

Scikit-image est une bibliothèque open source Python qui fonctionne avec les tableaux Numpy. Il implémente des algorithmes et des utilitaires destinés à la recherche, à l'éducation et aux applications industrielles.

2.2.6 SciPy



FIGURE 2.13 – SciPy

La librairie SciPy est un paquet contenant de nombreuses boîtes à outils consacrées aux méthodes de calcul scientifique. Ses différents sous-modules correspondent à différentes applications scientifiques, comme les méthodes d'interpolation, d'intégration, d'optimisation, de traitement d'images, de statistiques, de fonctions mathématiques spéciales, etc.

SciPy est dans la même catégorie de bibliothèques standards de calcul scientifique contenant, la GSL (GNU Scientific Library for C and C++), ou les boîtes à outils de Matlab. SciPy est la librairie à utiliser en Python pour les routines scientifiques ; elle fonctionne parfaitement sur les tableaux ou matrices NumPy de telle sorte que NumPy et SciPy peuvent interagir ensemble.

2.2.7 Dlib et FaceRecognition



FIGURE 2.14 – Dlib

Dlib est une bibliothèque logicielle multiplateforme à usage général écrite dans le langage de programmation C++. Sa conception est fortement influencée par les idées de la conception par contrat et par l'ingénierie logicielle basée sur les composants.

FaceRecognition est un modèle construit à l'aide Dlib. Le modèle a une précision qui peut atteindre 99,38%.

2.2.8 OverPass

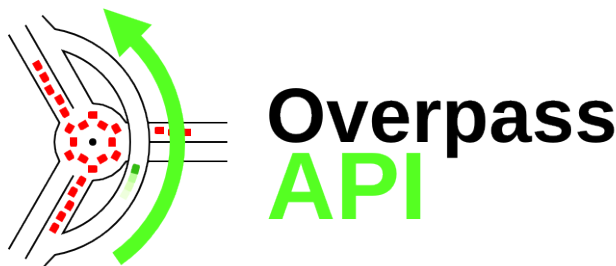


FIGURE 2.15 – OverPass

L'API Overpass [6] (anciennement connu sous OSM Server Side Scripting ou OSM3S avant 2011) est une API en lecture seule qui distribue des parties sélectionnées des données cartographiques OSM. Il s'agit d'une base de données sur le Web : le client envoie une requête à l'API et récupère l'ensemble de données qui correspond à la requête.

2.2.9 Pillow



FIGURE 2.16 – Pillow

Pillow est une bibliothèque de traitement d'image, qui est un fork et successeur du projet PIL (Python Imaging Library). Elle est conçue de manière à offrir un accès rapide aux données contenues dans une image, et offre un support pour différents formats de fichiers à savoir : PPM, PNG, JPEG, GIF, TIFF et BMP.

Pillow dispose de capacités de traitement d'images relativement puissantes, et a pour but d'offrir une solide base à toute application générale de traitement d'images.

2.3 Systèmes de sécurité et assistance à la conduite

Ce projet vise à réaliser deux systèmes indépendants en terme de fonctionnement, mais liés en termes de ressources. Le but est de créer les deux systèmes de façon optimisé vu les ressources matérielles relativement limités dont nous faisons possession.

2.3.1 Système de sécurité par reconnaissance faciale et suivi par GPS

Le premier système concerne un mécanisme de sécurité pour déverrouiller le véhicule en question.

Le propriétaire du véhicule enregistre dans la base de données du programme des photos de lui pour permettre à l'algorithme de reconnaissance faciale, d'apprendre son visage. Seul les personnes dont les visages sont reconnus par l'algorithme de reconnaissance sont capables de déverrouiller le véhicule. En cas de détection d'un visage non reconnu, le propriétaire du véhicule serait notifié par un email contenant la capture avec le visage de la personne non reconnu ainsi que les coordonnées de la position du véhicule. Ce système mobilise les deux module (Caméra et GPS) pour satisfaire les fonctionnalités citées précédemment.

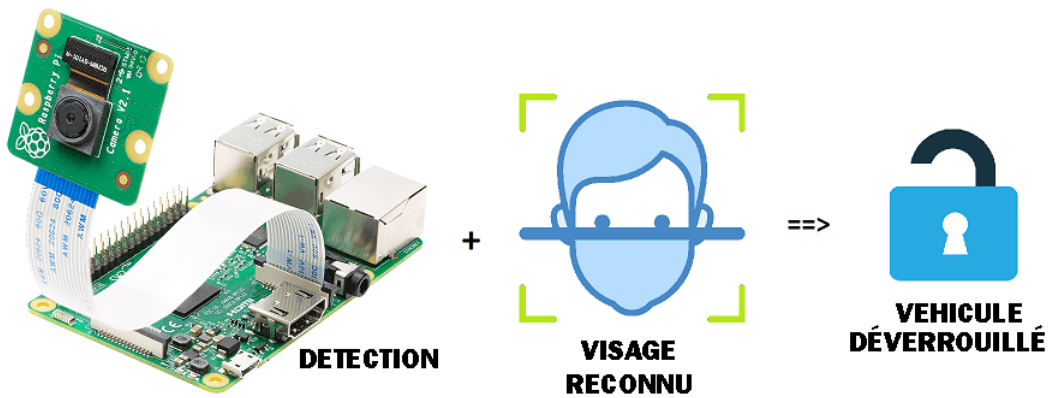


FIGURE 2.17 – Procédure de déverrouillage par reconnaissance faciale



FIGURE 2.18 – Mécanisme de sécurité

2.3.2 Système d'assistance à la conduite

Ce deuxième système concerne la détection et l'enregistrement des panneaux de circulation ainsi que la récupération de la vitesse maximale.

Le module caméra est maintenant utilisé comme une Dashcam, la détection des panneaux entraîne l'enregistrement du dernier panneau de circulation détecté. Les données du module GPS sont également mobilisé afin d'estimer une vitesse maximale pour la route emprunté par le conducteur.

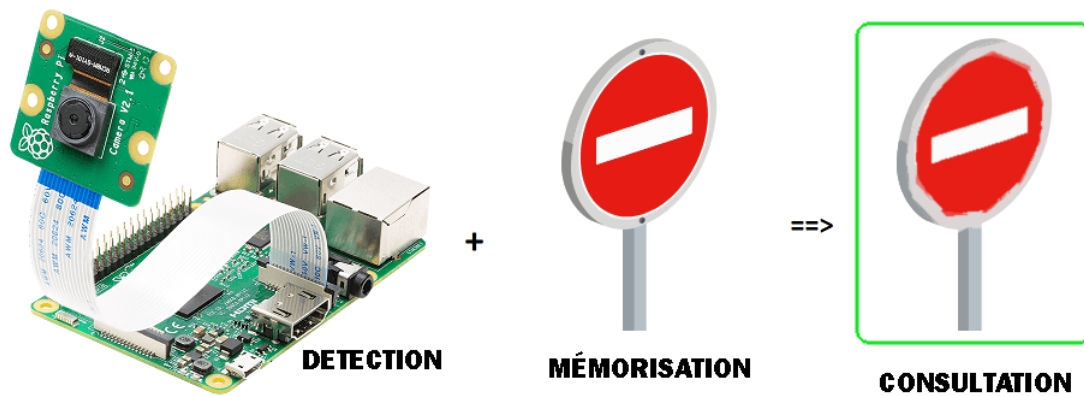


FIGURE 2.19 – Détection et mémorisation des panneaux de circulation

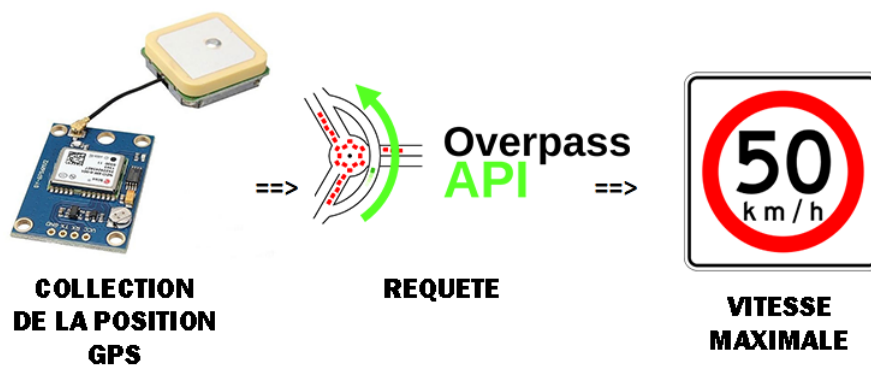


FIGURE 2.20 – Détection de la vitesse maximale

2.3.3 Technologies utilisées

Détection et reconnaissance faciale

Détection : Méthode de Viola-Jones

La méthode Viola-Jones consiste à utiliser les descripteurs de Haar qui sont constitués de deux rectangles adjacents, l'un noir et l'autre blanc. Ces descripteurs sont superposés aux différentes régions de l'image à la recherche d'une zone de forte corrélation.

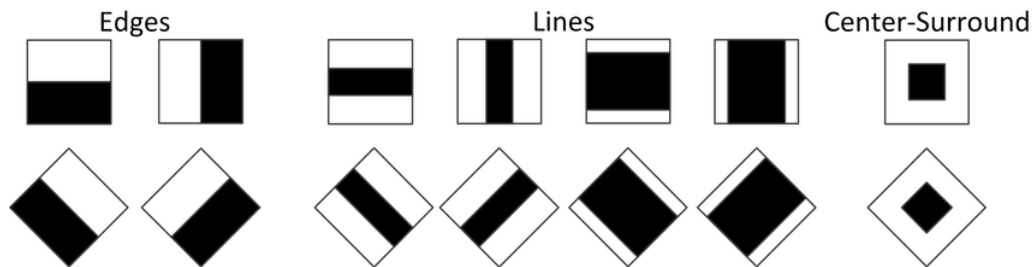


FIGURE 2.21 – Descripteurs de HAAR

L'algorithme détermine alors la ressemblance d'une zone de l'image aux descripteurs de Haar en soustrayant la moyenne des valeurs des pixels contenus dans la région noire à la moyenne des valeurs des pixels contenus dans la région blanche. La zone est retenue si cette différence est supérieure à un seuil. Le seuil utilisé est déterminé au cours de l'apprentissage par la méthode AdaBoost.

Il s'agit de la combinaison de plusieurs classificateurs peu performants (dits faibles), chacun assigné à un poids, pour en créer un beaucoup plus fiable (dit fort). La réponse de chaque classificateur (la région étudiée appartient au visage ou non) est peu fiable mais si le procédé est itératif, la réponse se fait de plus en plus précise au fur et à mesure du processus, d'où le nom de « cascade » de classificateurs.

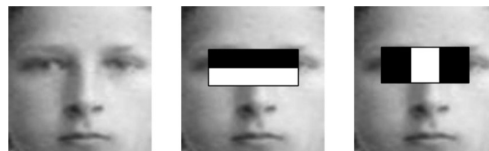


FIGURE 2.22 – Utilisation des descripteurs de HAAR

Cette méthode a une architecture simple qui fonctionne presque en temps réel sur le processeur. Pendant le traitement d'images reçues depuis la Picamera, il est indispensable d'exécuter une procédure de détection en permanence, ce qui nécessite de mobiliser un bon nombre de ressources au niveau matériel. C'est pourquoi elle a été mobilisée pour détecter les visages en temps réel pour être reconnu par la suite.

Reconnaissance : HOG

Histogram of Oriented Gradients permet de détecter l'orientation des pixels pour une case dans une grille donnée. L'orientation sera ensuite comparée à un descripteur d'objets. Ces orientations de pixels permettent de s'affranchir des informations superflues comme la couleur ou la luminosité.

L'idée importante derrière le descripteur HOG est que l'apparence et la forme locale d'un objet dans une image peuvent être décrites par la distribution de l'intensité du gradient ou la direction des contours.

Cette méthode est plus précise, c'est pourquoi elle est mobilisée pendant la phase d'apprentissage des nouveaux visages pour pouvoir par la suite les reconnaître.

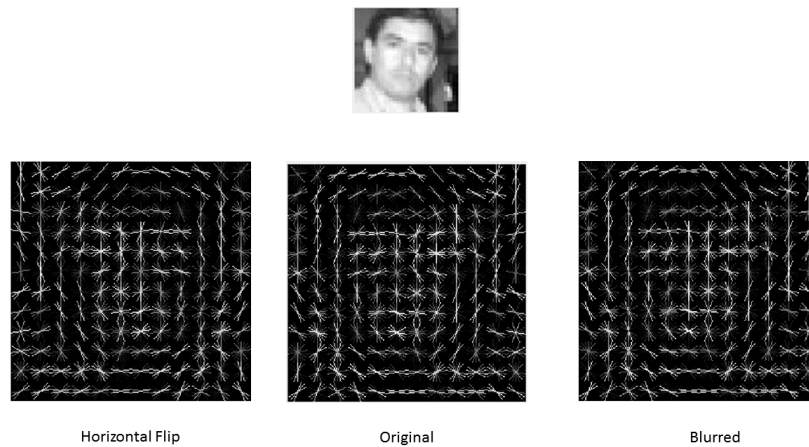


FIGURE 2.23 – Histogram of Oriented Gradients HOG

Détection des panneaux de circulation

En raison des contraintes matérielles il fallait réaliser un détecteur de panneaux de circulation avec une minimale consommation de ressources c'est pourquoi nous avons opté pour une détection de forme et filtrage par couleur.

La détection du panneau est entamée après un traitement d'image, permettant de déterminer les contours et de filtrer les couleurs indésirables. Ceci en exploitant les fonctionnalités offertes par OpenCV et SciPy.



FIGURE 2.24 – Panneaux de circulation récupérés après filtrage par contour et couleur.

3 Réalisation

3.1 Circuit utilisé

Avant d'entamer l'environnement logiciel du système, nous allons d'abord s'intéresser au circuit utilisé.

Le câblage des modules Picamera et GPS NEO 6Mv2 fut comme suit :

Le module GPS est connecté sur les entrées du GPIO à travers PIN 1(3,3 V), PIN 10(GPIO 15), PIN 8(GPIO 14), PIN 6(GND)

La Picamera est attachée sur le port caméra série (CSI).

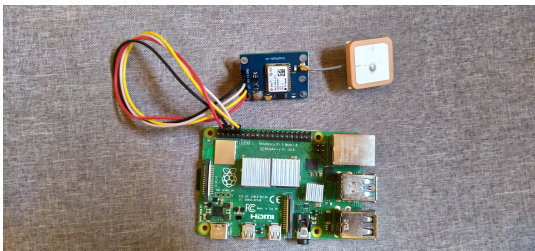


FIGURE 3.1 – Branchement des modules.

Un circuit complet du système, et attaché à l'ordinateur Raspberry Pi, figure dans l'image suivante.

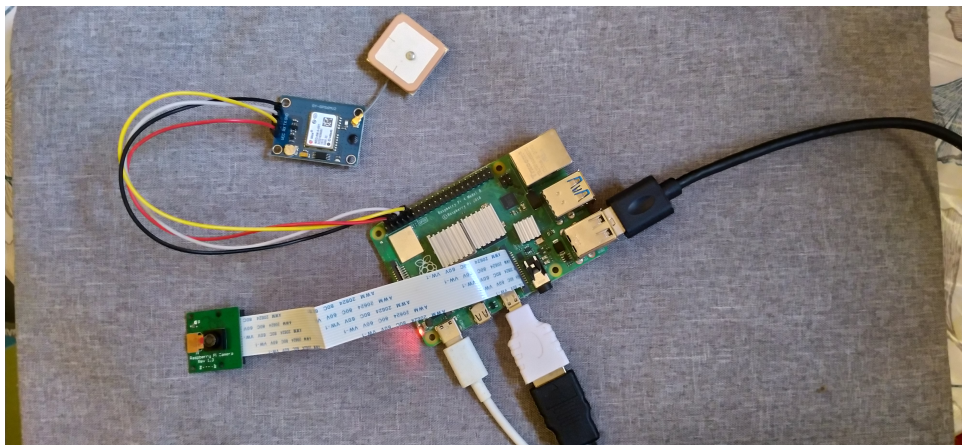


FIGURE 3.2 – Circuit complet

3.1.1 Configuration du module GPS

Le module GPS NEO 6Mv2 nécessite une configuration sur les ports du GPIO du Raspberry, après configuration, le module nécessite un temps de démarrage pour récupérer les données auprès des satellites. Un premier lancement du module est présent dans la figure suivante.

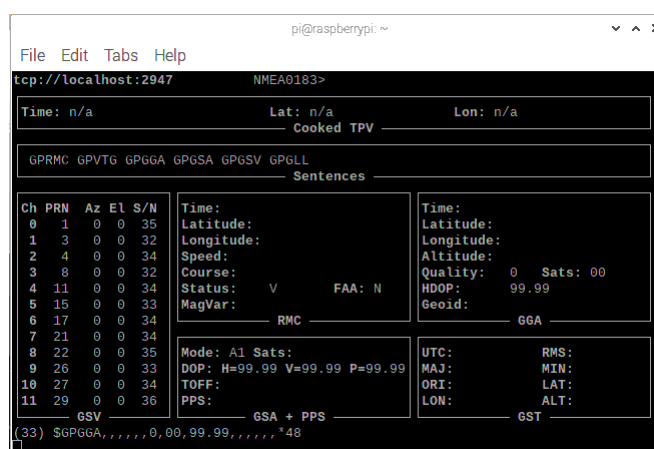


FIGURE 3.3 – Lancement du module GPS

Sur le tableau à gauche, figurent les satellites dont les signaux sont visibles par le module. Certaines informations concernant ces satellites sont également indiquées sur le même tableau, à savoir le S/R ou SNR (Signal to noise ratio) qui présente le rapport signal sur bruit en (dB-Hz). Après un certain temps de collection des données, le module demande un 3D Fix pour collecter les informations de localisation auprès des satellites. Cette demande peut prendre de 30 minutes jusqu'à 10 heures pendant le premier démarrage et afin d'attribuer le First Fix au module. L'image suivante présente le First Fix pour le module GPS NEO 6Mv2.

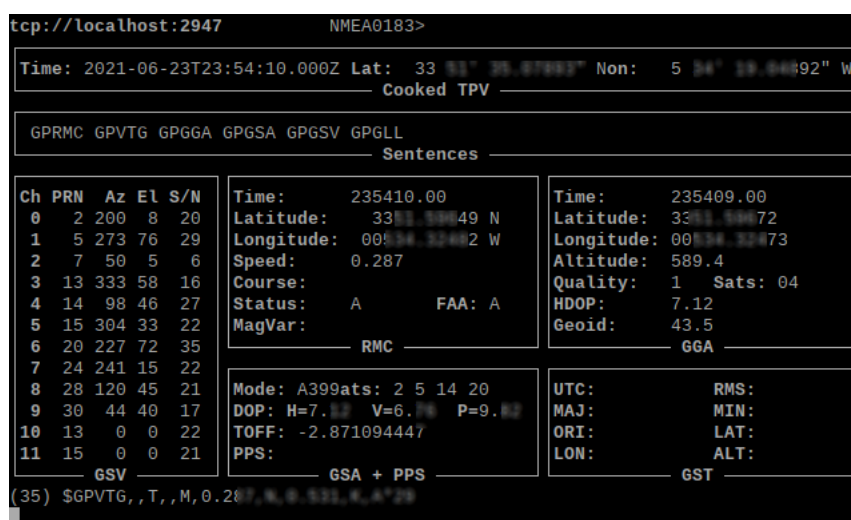
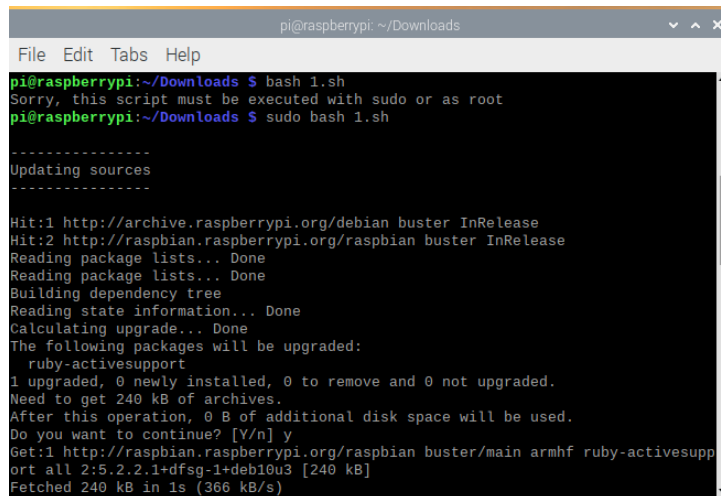


FIGURE 3.4 – First Fix

3.2 Installation des bibliothèques et dépendances

Afin d'installer OpenCV, qui est la bibliothèque utilisée principalement pour faire un traitement d'image, ainsi que les éventuelles tâches de détection et de reconnaissance, il était nécessaire d'installer d'abord certaines dépendances en termes logiciel.



```

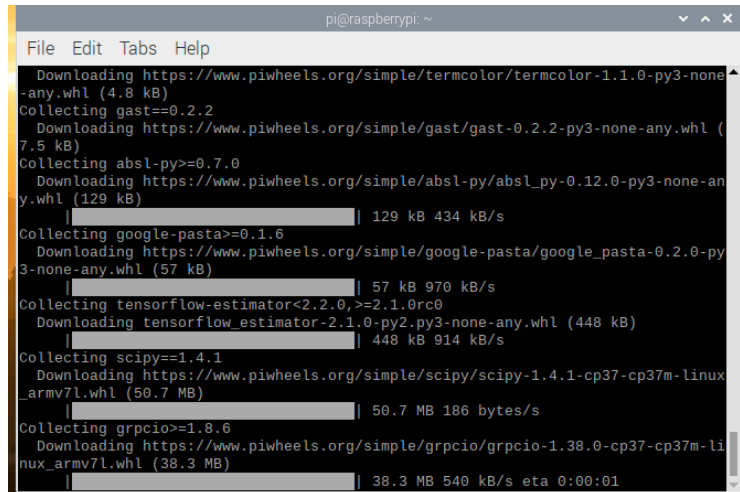
pi@raspberrypi: ~/Downloads
File Edit Tabs Help
pi@raspberrypi:~/Downloads $ bash 1.sh
Sorry, this script must be executed with sudo or as root
pi@raspberrypi:~/Downloads $ sudo bash 1.sh

-----
Updating sources
-----

Hit:1 http://archive.raspberrypi.org/debian buster InRelease
Hit:2 http://raspbian.raspberrypi.org/raspbian buster InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  ruby-activestorage
1 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 240 kB of archives.
After this operation, 0 B of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://raspbian.raspberrypi.org/raspbian buster/main armhf ruby-activestorage
all 2:5.2.2.1+dfsg-1+deb10u3 [240 kB]
Fetched 240 kB in 1s (366 kB/s)

```

FIGURE 3.5 – Installation des dépendances OpenCV



```

pi@raspberrypi: ~
File Edit Tabs Help
Downloading https://www.piwheels.org/simple/termcolor/termcolor-1.1.0-py3-none-any.whl (4.8 kB)
Collecting gast==0.2.2
Downloading https://www.piwheels.org/simple/gast/gast-0.2.2-py3-none-any.whl (7.5 kB)
Collecting absl-py>=0.7.0
Downloading https://www.piwheels.org/simple/absl-py/absl_py-0.12.0-py3-none-any.whl (129 kB)
Collecting google-pasta>=0.1.6
Downloading https://www.piwheels.org/simple/google-pasta/google_pasta-0.2.0-py3-none-any.whl (57 kB)
Collecting tensorflow-estimator<2.2.0,>=2.1.0rc0
Downloading tensorflow_estimator-2.1.0-py2.py3-none-any.whl (448 kB)
Collecting scipy==1.4.1
Downloading https://www.piwheels.org/simple/scipy/scipy-1.4.1-cp37m-linux_armv7l.whl (50.7 MB)
Collecting grpcio>=1.8.6
Downloading https://www.piwheels.org/simple/grpcio/grpcio-1.38.0-cp37m-linux_armv7l.whl (38.3 MB)

```

FIGURE 3.6 – Installation des dépendances OpenCV

L'étape suivante consiste à construire OpenCV depuis les sources téléchargées.

```

pi@raspberrypi: ~/Downloads
File Edit Tabs Help

opencv.zip          [          <=>          ] 87.55M 3.03MB/s in 57s

2021-06-10 01:49:44 (1.54 MB/s) - 'opencv.zip' saved [91806599]

pi@raspberrypi:~/Downloads $ wget -O opencv.zip https://github.com/opencv/opencv_contrib/archive/4.1.0.zip
--2021-06-10 01:50:13-- https://github.com/opencv/opencv_contrib/archive/4.1.0.zip
Resolving github.com (github.com)... 140.82.121.4
Connecting to github.com (github.com)[140.82.121.4]:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://codeload.github.com/opencv/opencv_contrib/zip/4.1.0 [following]
--2021-06-10 01:50:14-- https://codeload.github.com/opencv/opencv_contrib/zip/4.1.0
Resolving codeload.github.com (codeload.github.com)... 140.82.121.9
Connecting to codeload.github.com (codeload.github.com)[140.82.121.9]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/zip]
Saving to: 'opencv.zip'

opencv.zip          [          <=>          ] 30.62M 2.70MB/s

```

FIGURE 3.7 – Téléchargement des sources OpenCV

```

pi@raspberrypi:~/opencv_contrib-... [opencv_contrib-...]
CMakeOutput.log - Mousepad
File Edit Search View Document Help
Change Dir: /home/pi/Downloads/opencv-4.1.0/build/CMakeFiles/CMakeTmp

Run Build Command(s): /usr/bin/make cmTC_d1bbc/fast && /usr/bin/make -f CMakeFiles/
make[1]: Entering directory '/home/pi/Downloads/opencv-4.1.0/build/CMakeFiles/CMa
Building C object CMakeFiles/cmTC_d1bbc.dir/src.c.o
/usr/bin/cc -fsigned-char -W -Wall -Werror=return-type -Werror=non-virtual-dtor
/home/pi/Downloads/opencv-4.1.0/build/CMakeFiles/CMakeTmp/src.c:3:7: warning: no
void* test_func(void* data)
    ~~~~~^
Linking C executable cr
/usr/bin/cmake -E cmake File Edit Tab
/usr/bin/cc -fsigne make[1]: Leaving direc
Source file was:
#include <pthread.h>

void* test_func(void*
{
    return data;
}

[ 49%] Building CXX object modules/core/CMakeFiles/opencv_test_core.dir/test
arithmetic.cpp.o
[ 48%] Building CXX object modules/core/CMakeFiles/opencv_perf_core.dir/perf/ope
ncl/perf_channels.cpp.o
[ 48%] Building CXX object modules/imgproc/CMakeFiles/opencv_test_imgproc.dir/t
est/ocl/test_filters.cpp.o
[ 49%] Building CXX object modules/core/CMakeFiles/opencv_perf_core.dir/perf/ope
ncl/perf_dxt.cpp.o
[ 49%] Building CXX object modules/imgproc/CMakeFiles/opencv_perf_imgproc.dir/p
erf/opencv/perf_gfft.cpp.o
[ 49%] Building CXX object modules/core/CMakeFiles/opencv_perf_core.dir/perf/ope
ncl/perf_gemm.cpp.o
[ 49%] Building CXX object modules/imgproc/CMakeFiles/opencv_perf_imgproc.dir/p
erf/opencv/perf_houghlines.cpp.o
[ 49%] Building CXX object modules/imgproc/CMakeFiles/opencv_test_imgproc.dir/t
est/ocl/test_gfft.cpp.o
[ 49%] Building CXX object modules/core/CMakeFiles/opencv_perf_core.dir/perf/ope
ncl/perf_matop.cpp.o
[ 49%] Building CXX object modules/imgproc/CMakeFiles/opencv_perf_imgproc.dir/p
erf/opencv/perf_imgproc.cpp.o
[ 49%] Building CXX object modules/imgproc/CMakeFiles/opencv_test_imgproc.dir/t
est/ocl/test_histogram.cpp.o

```

FIGURE 3.8 – Compilation des sources OpenCV

Une fois la bibliothèque OpenCV installée, il fallait d'assurer que tout s'exécute correctement.

```

pi@raspberrypi: ~/Downloads
File Edit Tabs Help
Scanning dependencies of target pch_Generate_opencv_test_tracking
[ 24%] Generating perf_precomp.hpp
[ 24%] Generating test_precomp.hpp.gch/opencv_test_structured_light_RELEASE.gch
[ 24%] Generating test_precomp.hpp
[ 24%] Generating perf_precomp.hpp.gch/opencv_perf_tracking_RELEASE.gch
[ 24%] Generating precomp.hpp.gch/opencv_tracking_RELEASE.gch
[ 24%] Generating test_precomp.hpp.gch/opencv_test_tracking_RELEASE.gch
[ 24%] Built target pch_Generate_opencv_tracking
Scanning dependencies of target pch_Generate_opencv_videostab
[ 24%] Generating precomp.hpp
[ 24%] Generating precomp.hpp.gch/opencv_videostab_RELEASE.gch
[ 24%] Built target pch_Generate_opencv_test_structured_light
[ 24%] Built target pch_Generate_opencv_test_tracking
[ 24%] Built target pch_Generate_opencv_perf_tracking
Scanning dependencies of target pch_Generate_opencv_xfeatures2d
Scanning dependencies of target pch_Generate_opencv_test_videostab
Scanning dependencies of target pch_Generate_opencv_perf_xfeatures2d
[ 24%] Generating test_precomp.hpp
[ 24%] Generating precomp.hpp
[ 24%] Generating perf_precomp.hpp
[ 24%] Generating test_precomp.hpp.gch/opencv_test_videostab_RELEASE.gch
[ 24%] Generating perf_precomp.hpp.gch/opencv_perf_xfeatures2d_RELEASE.gch
[ 24%] Generating precomp.hpp.gch/opencv_xfeatures2d_RELEASE.gch

```

FIGURE 3.9 – Compilation des sources OpenCV

```

pi@raspberrypi: ~/caffe/caffe-ssd
File Edit Tabs Help
[ RUN      ] BenchmarkTest/0.TestTimerStart
[ OK       ] BenchmarkTest/0.TestTimerStart (0 ms)
[ RUN      ] BenchmarkTest/0.TestTimerStop
[ OK       ] BenchmarkTest/0.TestTimerStop (0 ms)
[ RUN      ] BenchmarkTest/0.TestTimerMilliseconds
[ OK       ] BenchmarkTest/0.TestTimerMilliseconds (300 ms)
[ RUN      ] BenchmarkTest/0.TestTimerSeconds
[ OK       ] BenchmarkTest/0.TestTimerSeconds (300 ms)
[-----] 5 tests from BenchmarkTest/0 (601 ms total)

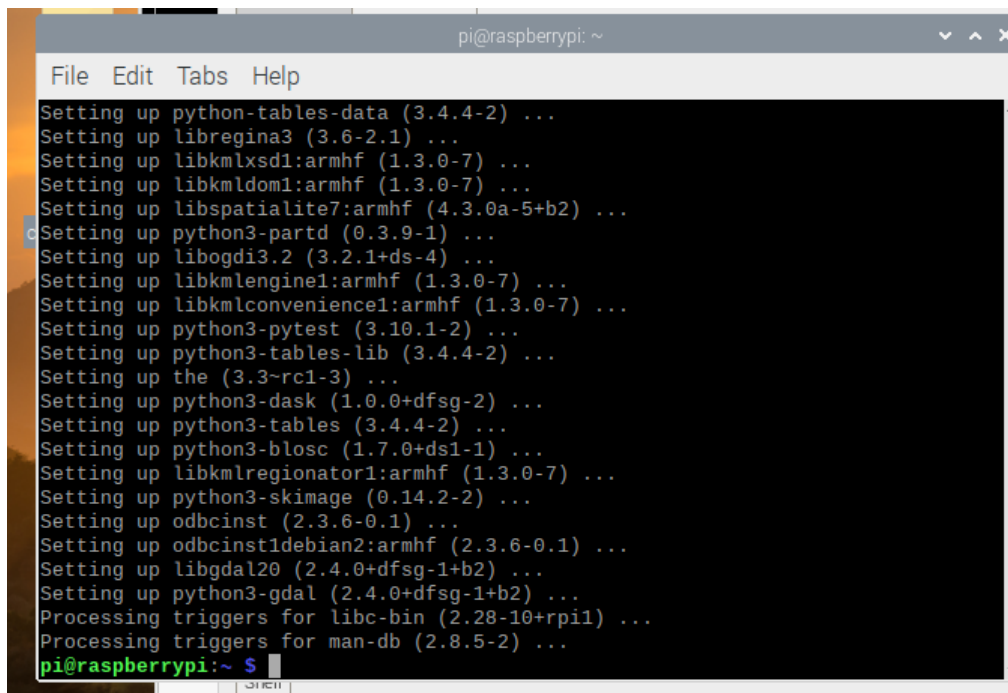
[-----] 1 test from MultinomialLogisticLossLayerTest/0, where TypeParam = float
[ RUN      ] MultinomialLogisticLossLayerTest/0.TestGradientCPU
[ OK       ] MultinomialLogisticLossLayerTest/0.TestGradientCPU (0 ms)
[-----] 1 test from MultinomialLogisticLossLayerTest/0 (0 ms total)

[-----] 4 tests from ContrastiveLossLayerTest/1, where TypeParam = caffe::CPUDevice<double>
[ RUN      ] ContrastiveLossLayerTest/1.TestGradient
[ OK       ] ContrastiveLossLayerTest/1.TestGradient (334 ms)
[ RUN      ] ContrastiveLossLayerTest/1.TestForward
[ OK       ] ContrastiveLossLayerTest/1.TestForward (1 ms)
[ RUN      ] ContrastiveLossLayerTest/1.TestGradientLegacy

```

FIGURE 3.10 – Test de vérification OpenCV

Le processus se poursuit par l'installation des dépendances nécessaires au bon fonctionnement d'OpenCV, TensorFlow, et Keras

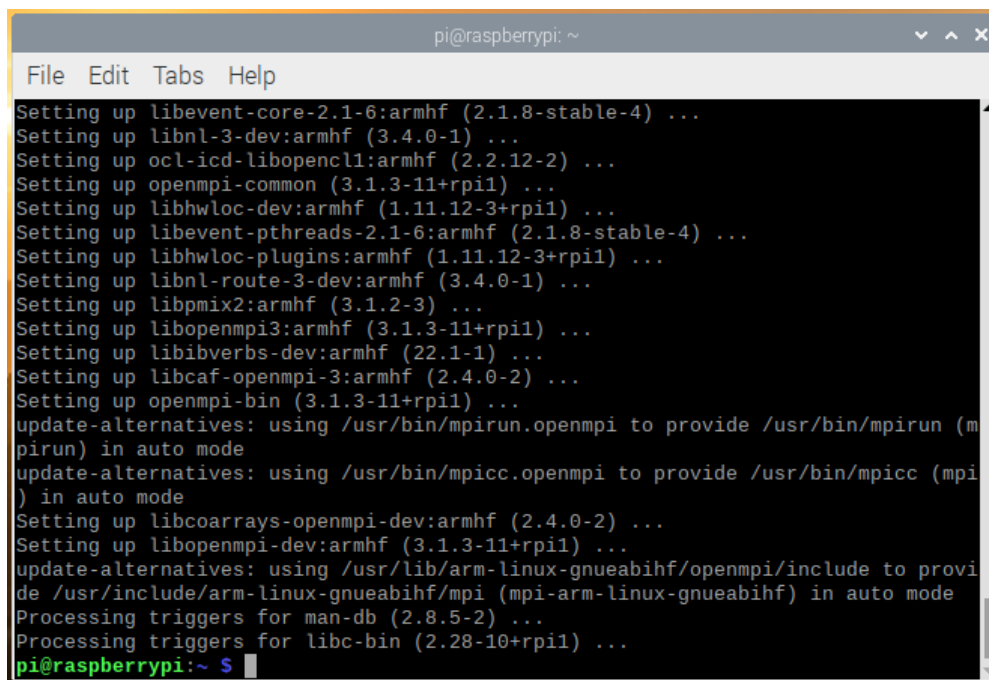


```

pi@raspberrypi: ~
File Edit Tabs Help
Setting up python-tables-data (3.4.4-2) ...
Setting up libregina3 (3.6-2.1) ...
Setting up libkmlxsd1:armhf (1.3.0-7) ...
Setting up libkmlcore1:armhf (1.3.0-7) ...
Setting up libspatialite7:armhf (4.3.0a-5+b2) ...
Setting up python3-partd (0.3.9-1) ...
Setting up libogdi3.2 (3.2.1+ds-4) ...
Setting up libkmlengine1:armhf (1.3.0-7) ...
Setting up libkmlconvenience1:armhf (1.3.0-7) ...
Setting up python3-pytest (3.10.1-2) ...
Setting up python3-tables-lib (3.4.4-2) ...
Setting up the (3.3-rc1-3) ...
Setting up python3-dask (1.0.0+dfsg-2) ...
Setting up python3-tables (3.4.4-2) ...
Setting up python3-blosc (1.7.0+ds1-1) ...
Setting up libkmlregionator1:armhf (1.3.0-7) ...
Setting up python3-skimage (0.14.2-2) ...
Setting up odbcinst (2.3.6-0.1) ...
Setting up odbcinst1debian2:armhf (2.3.6-0.1) ...
Setting up libgdal20 (2.4.0+dfsg-1+b2) ...
Setting up python3-gdal (2.4.0+dfsg-1+b2) ...
Processing triggers for libc-bin (2.28-10+rpil) ...
Processing triggers for man-db (2.8.5-2) ...
pi@raspberrypi:~ $

```

FIGURE 3.11 – Installation des dépendances TensorFlow et Keras



```

pi@raspberrypi: ~
File Edit Tabs Help
Setting up libevent-core-2.1-6:armhf (2.1.8-stable-4) ...
Setting up libnl-3-dev:armhf (3.4.0-1) ...
Setting up ocl-icd-libopencl1:armhf (2.2.12-2) ...
Setting up openmpi-common (3.1.3-11+rpil) ...
Setting up libhwloc-dev:armhf (1.11.12-3+rpil) ...
Setting up libevent-pthreads-2.1-6:armhf (2.1.8-stable-4) ...
Setting up libhwloc-plugins:armhf (1.11.12-3+rpil) ...
Setting up libnl-route-3-dev:armhf (3.4.0-1) ...
Setting up libpmix2:armhf (3.1.2-3) ...
Setting up libopenmpi3:armhf (3.1.3-11+rpil) ...
Setting up libibverbs-dev:armhf (22.1-1) ...
Setting up libcaf-openmpi-3:armhf (2.4.0-2) ...
Setting up openmpi-bin (3.1.3-11+rpil) ...
update-alternatives: using /usr/bin/mpirun.openmpi to provide /usr/bin/mpirun (m
pirun) in auto mode
update-alternatives: using /usr/bin/mpicc.openmpi to provide /usr/bin/mpicc (mpi
) in auto mode
Setting up libcoarrays-openmpi-dev:armhf (2.4.0-2) ...
Setting up libopenmpi-dev:armhf (3.1.3-11+rpil) ...
update-alternatives: using /usr/lib/arm-linux-gnueabi/hf/openmpi/include to provi
de /usr/include/arm-linux-gnueabi/hf/mpi (mpi-arm-linux-gnueabi/hf) in auto mode
Processing triggers for man-db (2.8.5-2) ...
Processing triggers for libc-bin (2.28-10+rpil) ...
pi@raspberrypi:~ $

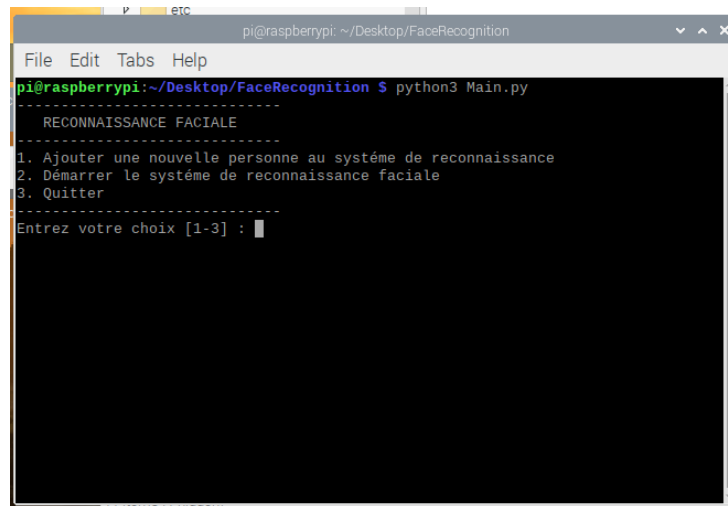
```

FIGURE 3.12 – Installation des dépendances TensorFlow et Keras

3.3 Système de détection et reconnaissance faciale

Le système est divisé en plusieurs fichiers responsables pour chaque groupe de procédures nécessaires à l'exécution correcte du système.

On commence par exécuter le fichier "Main.py" contenant la fonction principale, initialisant le bon environnement d'exécution du système.

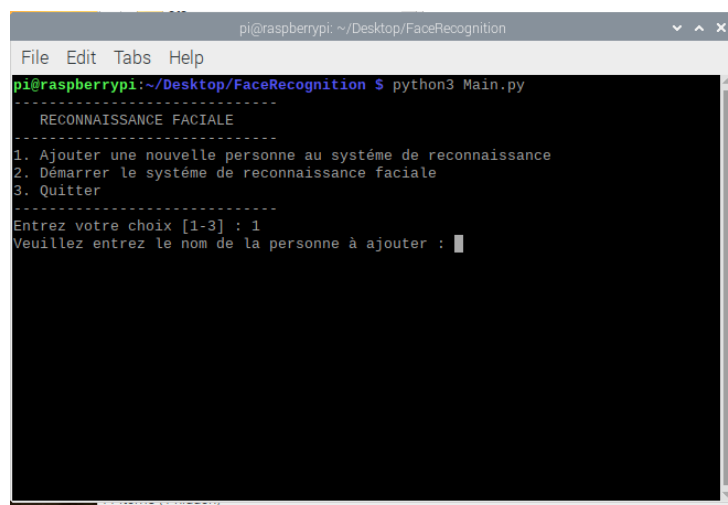


```
pi@raspberrypi: ~/Desktop/FaceRecognition
File Edit Tabs Help
pi@raspberrypi:~/Desktop/FaceRecognition $ python3 Main.py
-----
RECONNAISSANCE FACIALE
-----
1. Ajouter une nouvelle personne au système de reconnaissance
2. Démarrer le système de reconnaissance faciale
3. Quitter
-----
Entrez votre choix [1-3] : █
```

FIGURE 3.13 – Execution du système de reconnaissance faciale

La première opération consiste d'ajouter une nouvelle personne à reconnaître plus tard par le système.

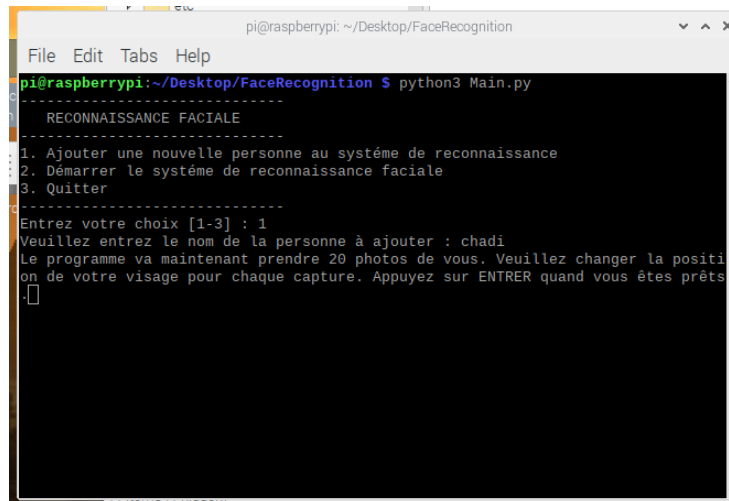
Pour cela nous choisissons la première option du menu affiché : "Ajouter une nouvelle personne au système de reconnaissance".



```
pi@raspberrypi: ~/Desktop/FaceRecognition
File Edit Tabs Help
pi@raspberrypi:~/Desktop/FaceRecognition $ python3 Main.py
-----
RECONNAISSANCE FACIALE
-----
1. Ajouter une nouvelle personne au système de reconnaissance
2. Démarrer le système de reconnaissance faciale
3. Quitter
-----
Entrez votre choix [1-3] : 1
Veuillez entrer le nom de la personne à ajouter : █
```

FIGURE 3.14 – Demarrer l'ajout d'une nouvelle personne au système de reconnaissance.

Le système demande à l'utilisateur par la suite le nom de la personne en question, qu'il faut renseigner pour procéder à la prochaine étape.



```

pi@raspberrypi: ~/Desktop/FaceRecognition
File Edit Tabs Help
pi@raspberrypi:~/Desktop/FaceRecognition $ python3 Main.py
-----
RECONNAISSANCE FACIALE
-----
1. Ajouter une nouvelle personne au système de reconnaissance
2. Démarrer le système de reconnaissance faciale
3. Quitter
-----
Entrez votre choix [1-3] : 1
Veuillez entrer le nom de la personne à ajouter : chadi
Le programme va maintenant prendre 20 photos de vous. Veuillez changer la position
de votre visage pour chaque capture. Appuyez sur ENTRER quand vous êtes prêts
.

```

FIGURE 3.15 – Renseigner des informations sur la personne à ajouter au système.

Le système informe ensuite l'utilisateur sur l'étape à suivre : En mobilisant la Picamera, le système capture 20 photos du sujet en question puis détecte et enregistre les 20 visages à coder plus tard.

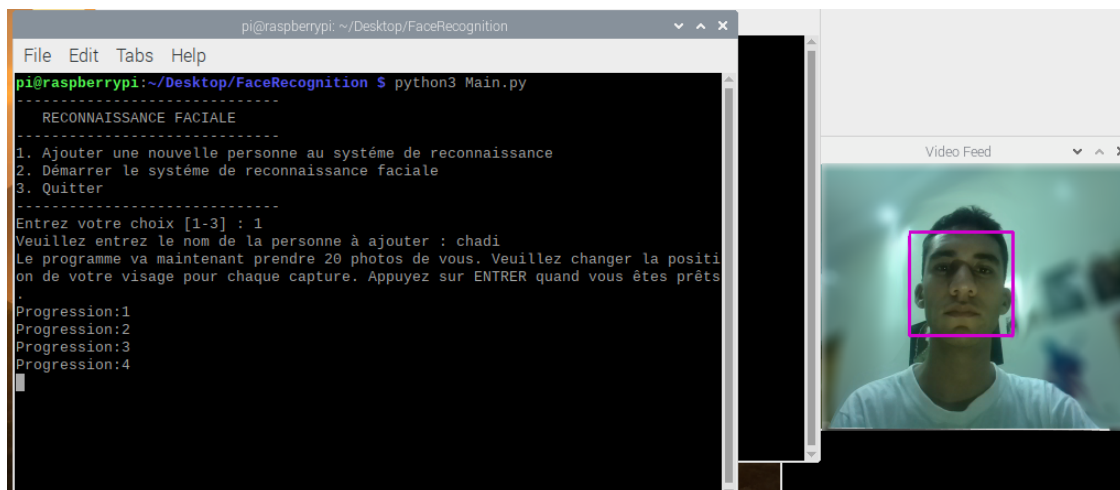
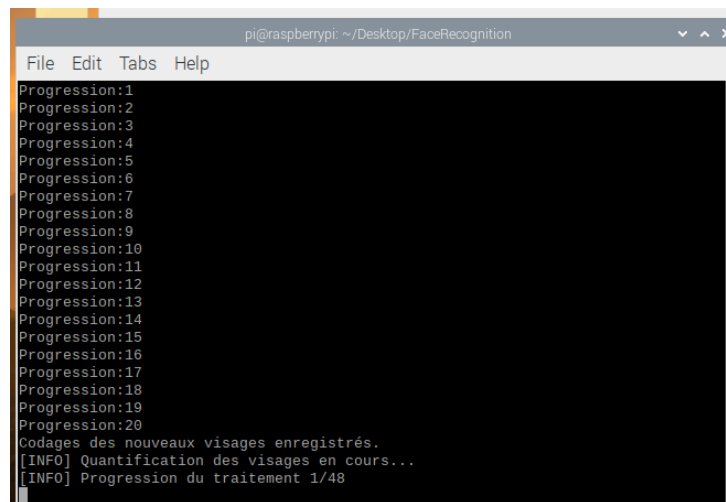


FIGURE 3.16 – Enregistrement du visage du sujet à travers la détection faciale.

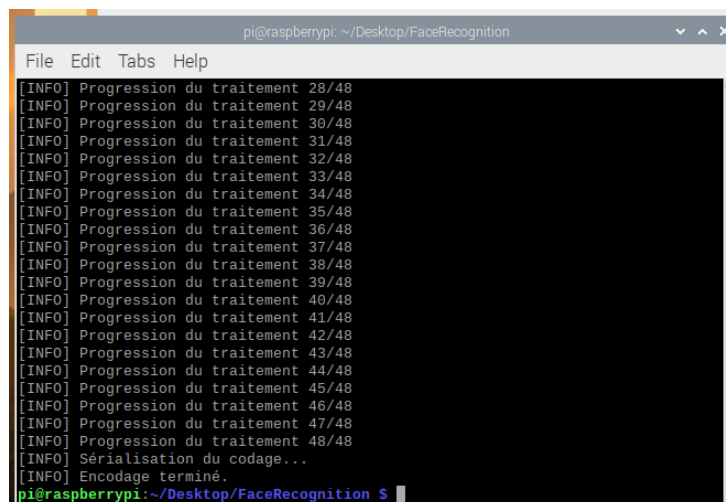
Une fois cette étape complétée, le système va poursuivre en lançant la procédure du codage des nouveaux visages.



```
pi@raspberrypi: ~/Desktop/FaceRecognition
File Edit Tabs Help
Progression:1
Progression:2
Progression:3
Progression:4
Progression:5
Progression:6
Progression:7
Progression:8
Progression:9
Progression:10
Progression:11
Progression:12
Progression:13
Progression:14
Progression:15
Progression:16
Progression:17
Progression:18
Progression:19
Progression:20
Codages des nouveaux visages enregistrés.
[INFO] Quantification des visages en cours...
[INFO] Progression du traitement 1/48
```

FIGURE 3.17 – Enregistrement des nouveaux visages.

L'opération peut prendre quelque minutes, pour permettre au Raspberry Pi de terminer les calculs nécessaires.



```
pi@raspberrypi: ~/Desktop/FaceRecognition
File Edit Tabs Help
[INFO] Progression du traitement 28/48
[INFO] Progression du traitement 29/48
[INFO] Progression du traitement 30/48
[INFO] Progression du traitement 31/48
[INFO] Progression du traitement 32/48
[INFO] Progression du traitement 33/48
[INFO] Progression du traitement 34/48
[INFO] Progression du traitement 35/48
[INFO] Progression du traitement 36/48
[INFO] Progression du traitement 37/48
[INFO] Progression du traitement 38/48
[INFO] Progression du traitement 39/48
[INFO] Progression du traitement 40/48
[INFO] Progression du traitement 41/48
[INFO] Progression du traitement 42/48
[INFO] Progression du traitement 43/48
[INFO] Progression du traitement 44/48
[INFO] Progression du traitement 45/48
[INFO] Progression du traitement 46/48
[INFO] Progression du traitement 47/48
[INFO] Progression du traitement 48/48
[INFO] Sérialisation du codage...
[INFO] Encodage terminé.
pi@raspberrypi:~/Desktop/FaceRecognition $
```

FIGURE 3.18 – Codage et sérialisation des nouvelles données.

Une fois cette étape est terminée, l'utilisateur peut redémarrer le programme pour démarrer cette fois le système de reconnaissance.

L'utilisateur redémarre le programme, cette fois en sélectionnant le deuxième choix pour "Démarrer la reconnaissance faciale", permettant de sécuriser le véhicule en question.

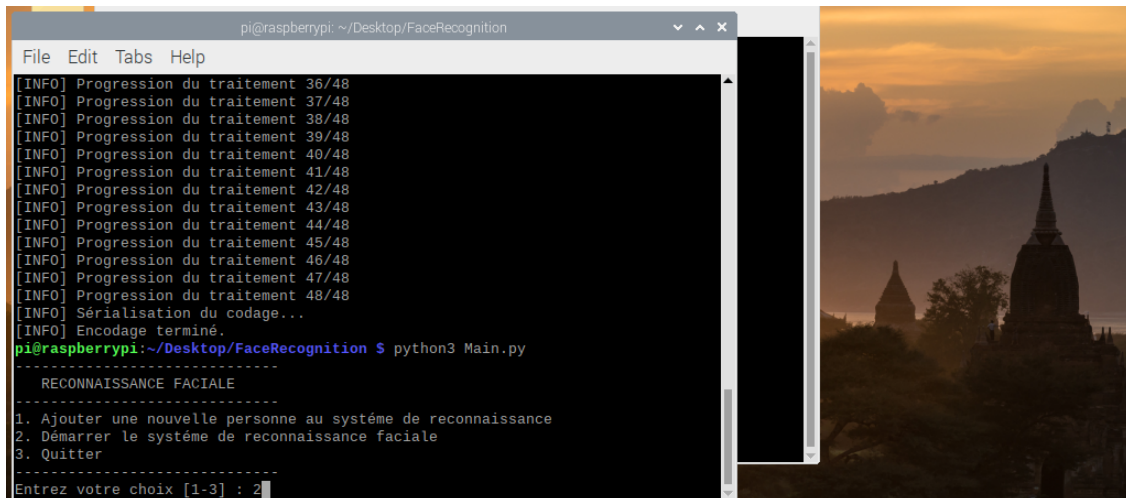


FIGURE 3.19 – Démarrage du système de reconnaissance faciale.

La détection et la reconnaissance des visages est, par la suite, exécutée en temps réel, et l'ensemble des faces des personnes regardant directement la Picamera seront localisés et comparés avec les données des visages précédemment enregistrés.

Un visage reconnu par le système ressemblera au suivant.

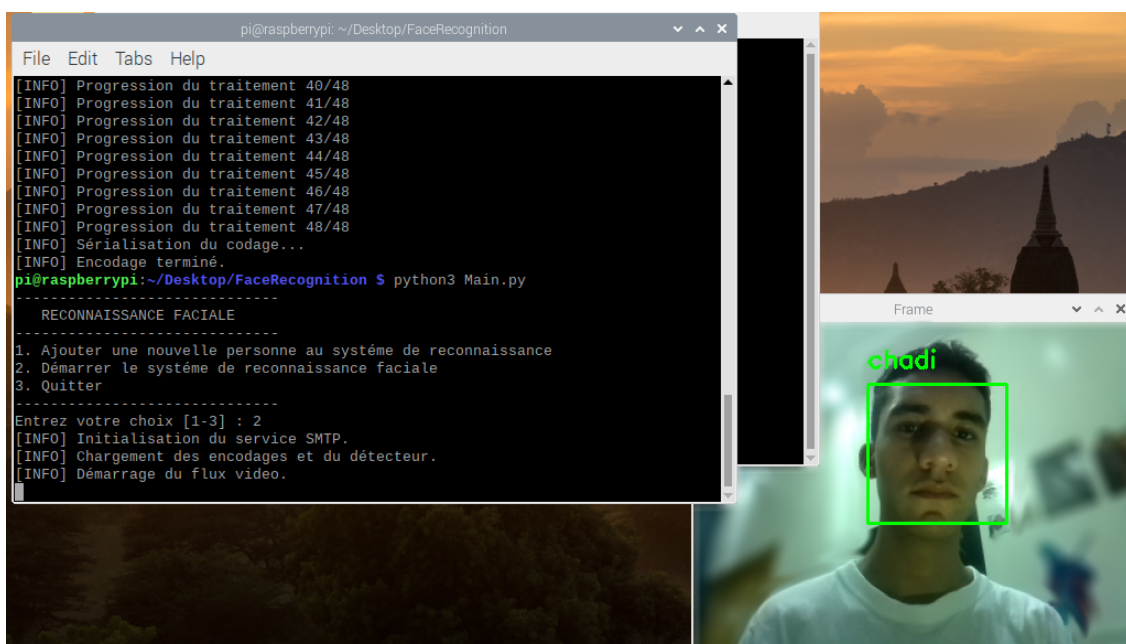


FIGURE 3.20 – Reconnaissance faciale.

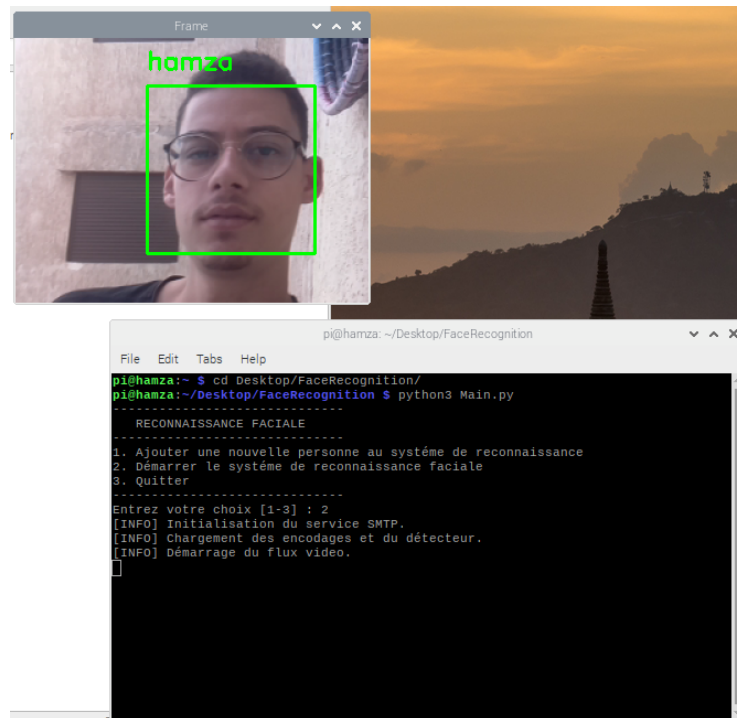


FIGURE 3.21 – Reconnaissance faciale.

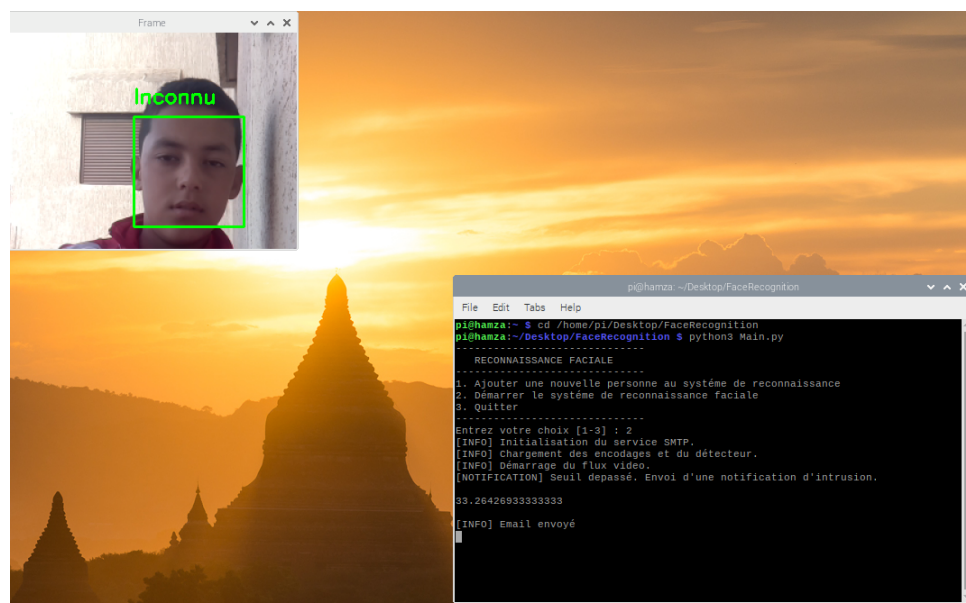


FIGURE 3.22 – Visage non reconnu détecté.

Si un visage non reconnu est détecté, le système, localise ce dernier, et prends un moment (Environ 10 sec) pour s'assurer que c'est le cas d'une tentative d'intrusion au véhicule, puis capture le visage de l'inconnu en question, ainsi que les coordonnées GPS, indiquant la position actuelle du véhicule, et envoie une notification d'intrusion au propriétaire par le biais d'un email.

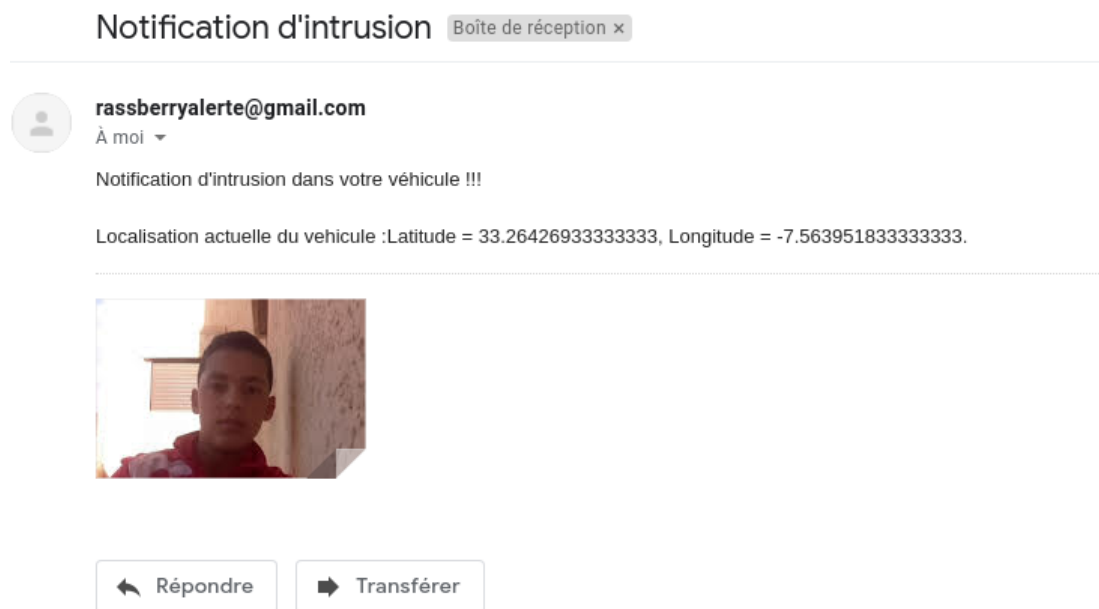


FIGURE 3.23 – Notification d'intrusion.

3.4 Système de détection des panneaux, et vitesse maximale.

Ce deuxième système entame deux procédures s'exécutant en parallèle.

3.4.1 Détection des panneaux de circulation

D'abord le flux video est issu de la Picamera en position de Dashcam et le système commence directement la détection est la mémorisation des panneaux de circulation.

Nous allons voir le fonctionnement du système avec les panneaux suivants, de formes et de couleurs différents.

L'API OpenCV permet d'exécuter un algorithme de détection des contours qui délimite les zones d'intérêt dans l'image, ensuite les différents filtres à couleur appliqués à l'image en entrée, issue de la Picamera, permettent de déterminer à travers des seuils s'il s'agit d'un potentiel panneau ou pas. La non conformité au contour et à la couleur permet de filtrer les faux positifs.

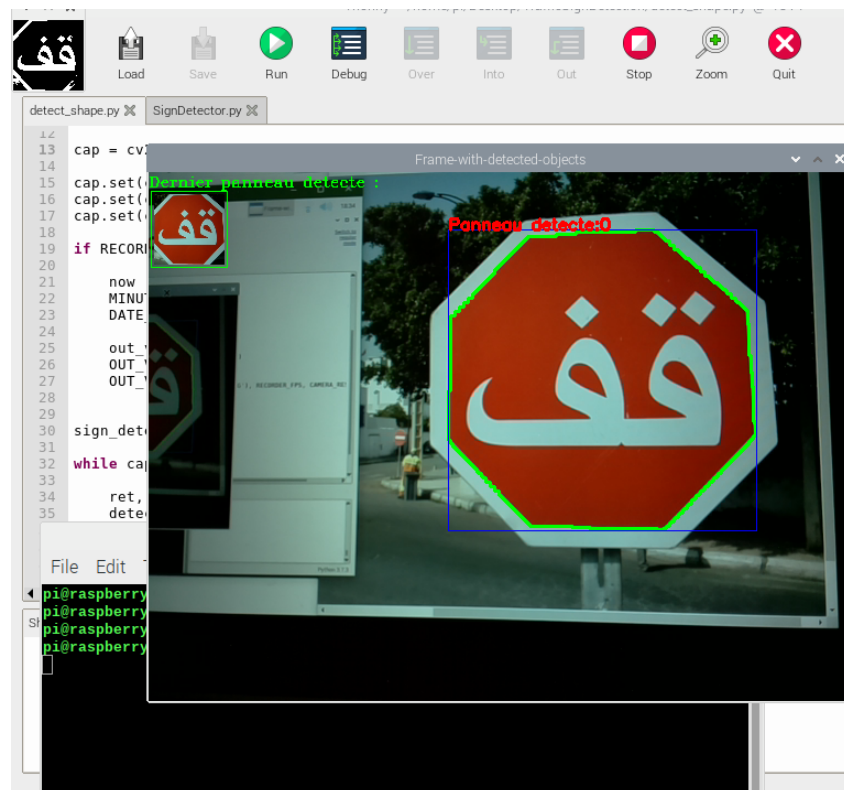


FIGURE 3.24 – Détection du panneau 'STOP'.



FIGURE 3.25 – Détection du panneau de limitation de vitesse.

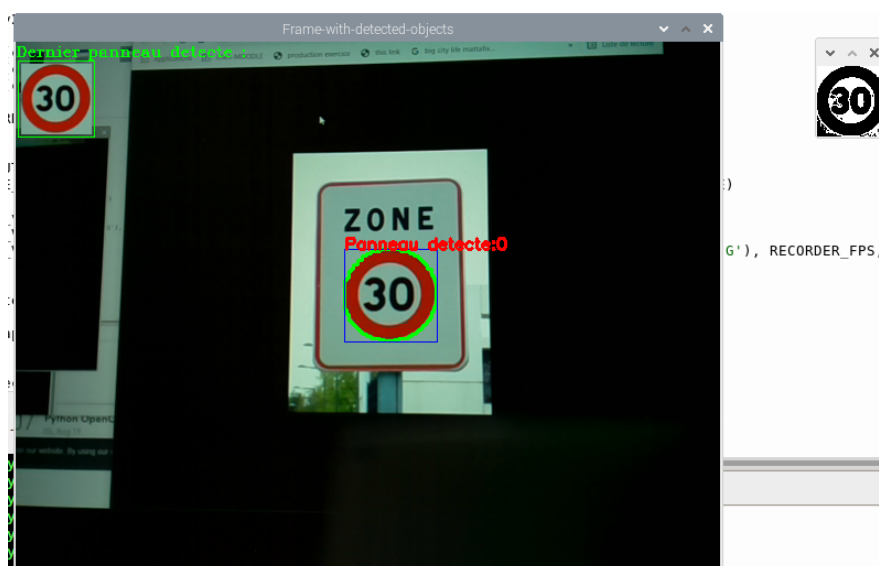


FIGURE 3.26 – Détection du panneau de limitation de vitesse.

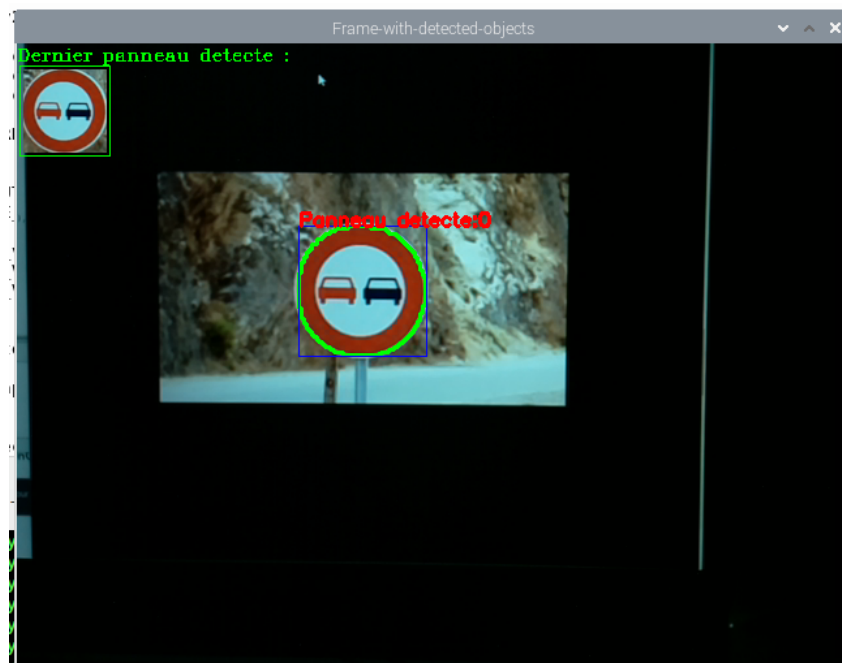


FIGURE 3.27 – Détection du panneau d'interdiction de doublage.



FIGURE 3.28 – Détection d'un panneau d'interdiction.

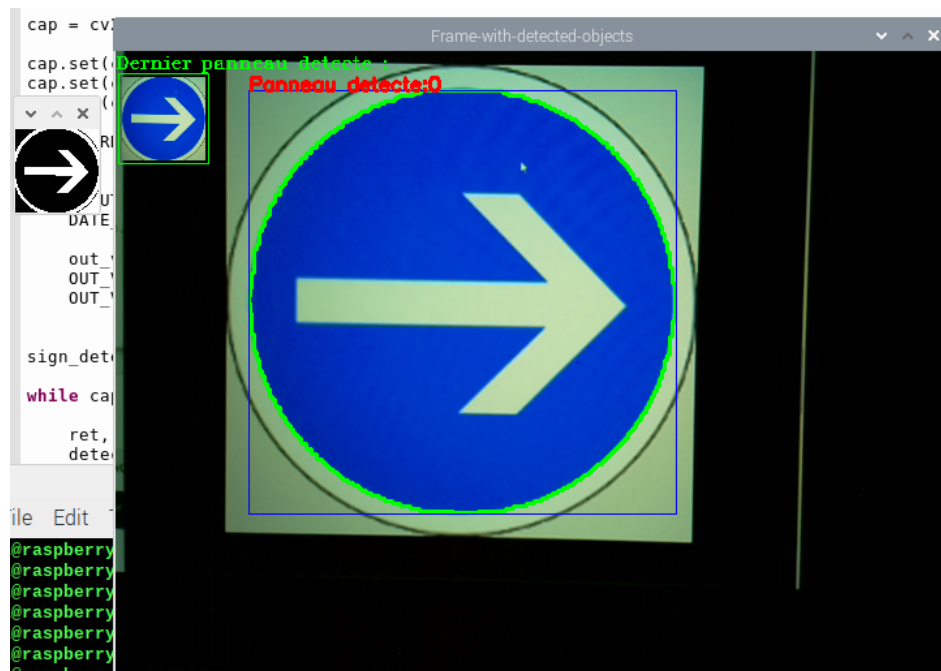


FIGURE 3.29 – Détection d'un panneau d'obligation.

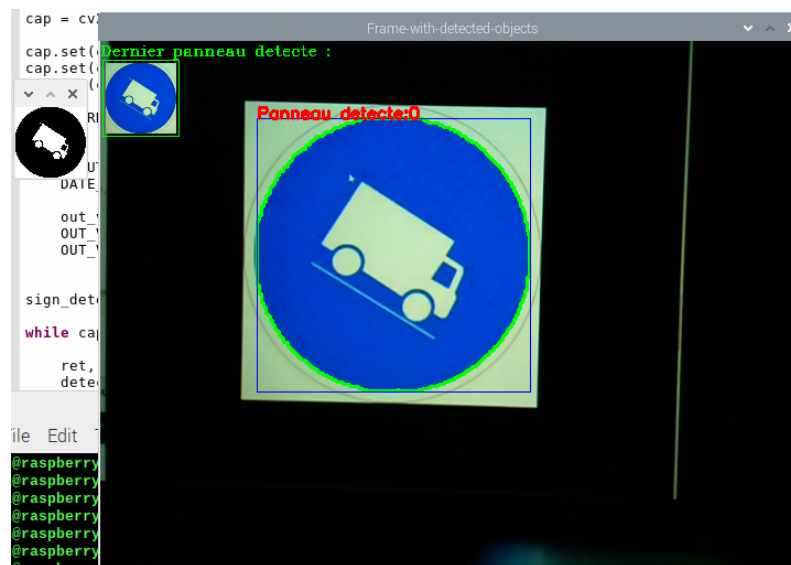


FIGURE 3.30 – Détection d'un panneau d'obligation.

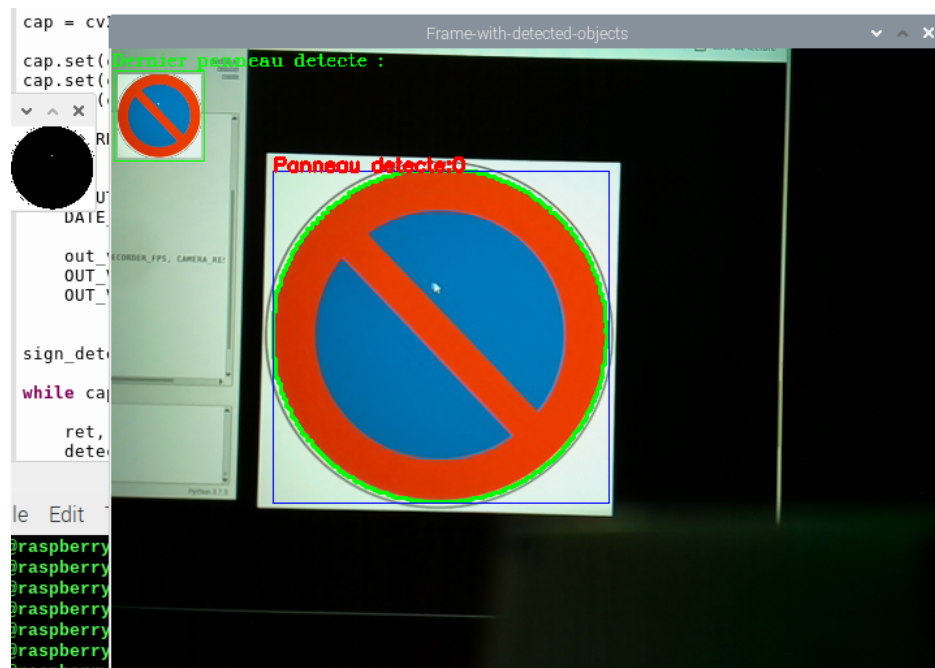


FIGURE 3.31 – Détection du panneau d’interdiction de stationnement.



FIGURE 3.32 – Détection du panneau de danger : Succession de virages’.

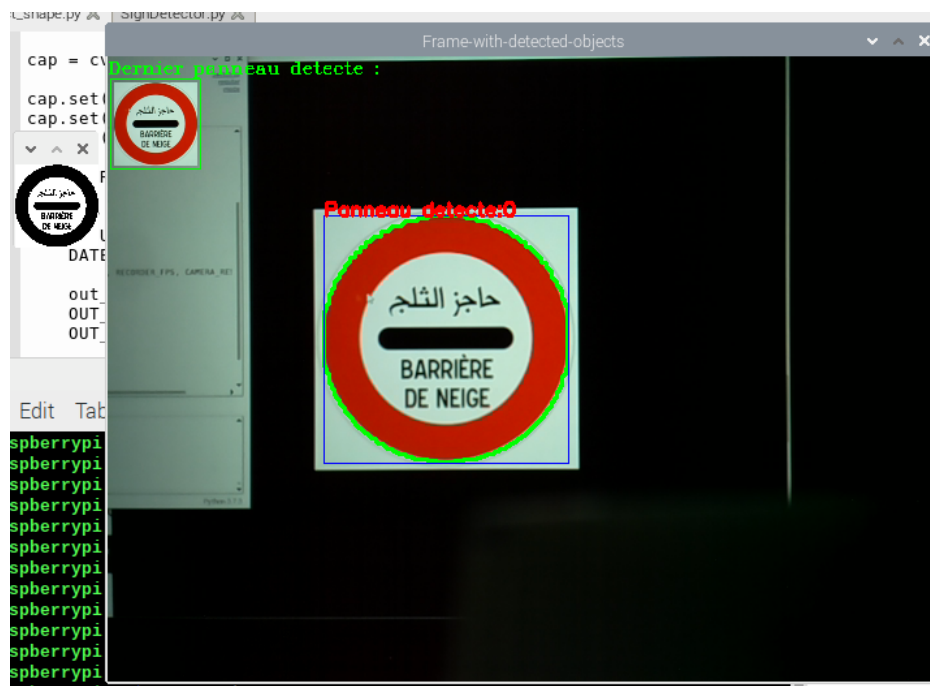


FIGURE 3.33 – Détection du panneau : 'Barrière de neige'.

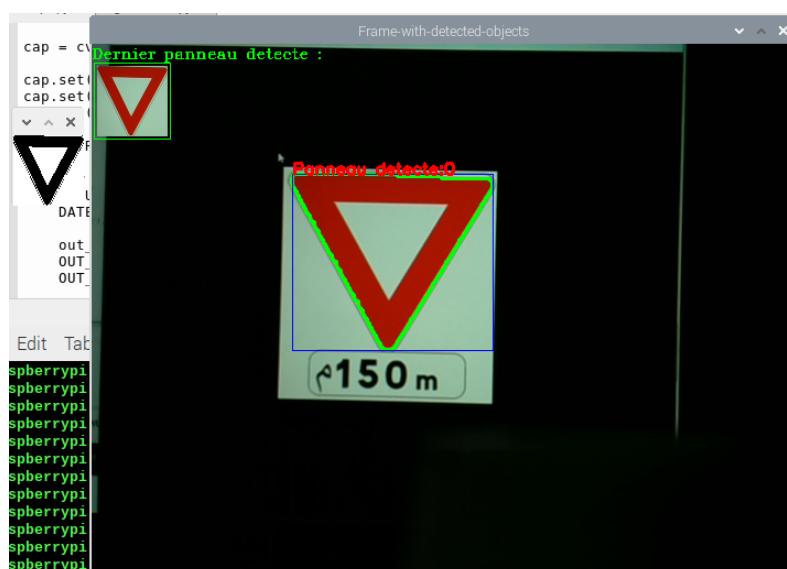


FIGURE 3.34 – Détection du panneau : 'Cédez le passage'.

Cette détection permet enfin, de garder en vue le dernier panneau de circulation détecté par le système, ceci dans le coin en haut à gauche de l'image courante.

3.4.2 Détermination de la vitesse maximale

Viens ensuite, la détermination de la vitesse maximale pour la position actuelle du véhicule.

À travers les coordonnées collectées depuis le module GPS, le système est capable de déterminer la vitesse maximale allouée dans localisation actuelle du véhicule, ceci permet d'afficher à l'utilisateur une valeur de vitesse maximale, actualisée à des intervalles de 20 secondes.



FIGURE 3.35 – Fonctionnement du système d'assistance à la conduite

La détection et mémorisation des panneaux de circulation toujours s'exécutant toujours en temps réel, est désormais accompagné des informations issus de la localisation du véhicule, le nom de la route parcourue, et la vitesse maximale allouée.

La détection des panneaux donne une sortie vidéo avec un taux d'images par seconde proche du 7 (Moyenne de 7 FPS).

L'actualisation des données de localisation se fait, quant à elle, à des intervalles de 20 secondes pour économiser l'utilisation des ressources.

3.5 Résultats et futur du projet

3.5.1 Performance

Même avec la puissance impressionnante du Raspberry Pi, la détection et la reconnaissance d'objets fut une tâche qui mène l'ordinateur à une utilisation énorme de ressources pour compléter les calculs nécessaires.

Par conséquent la reconnaissance des panneaux de circulation fut une tâche entamée par le projet mais qui n'a pas vu le jour en raison de l'incapacité du Raspberry de se maintenir en fonction pour une longue durée et pour le grand nombre de panneaux de circulation qui existent. Le modèle Keras utilisé pouvait effectivement reconnaître des panneaux après leur détection mais ne pouvait fonctionner correctement pour un grand nombre d'images auxquelles il doit s'entraîner.

Dans l'avenir de ce projet une possible solution serait d'opter pour des techniques IOT pour résoudre le problème des calculs lourds.

Les images enregistrées par la Picamera, seront traitées localement pour la détection (Phase réalisée dans ce projet), mais la reconnaissance serait performée par un serveur distant qui reçoit les images des panneaux détectés et renvoi en réponse les informations à propos du panneau reconnu.

Quant à la détection et la reconnaissance faciale, le système s'avère être équitabement performant avec une sortie en image qui atteint les 7 images par secondes (7 FPS). Il est tout de même convenable d'affirmer qu'il est possible, soit pour la reconnaissance faciale, soit pour la détection des panneaux, d'obtenir un rendement encore plus robuste grâce à des améliorations au niveau matériel.

Neural Compute Sticks

Afin d'atteindre des niveaux de performance élevés, et permettre plus de calculs potentiellement ouvrant une voie pour adopter une reconnaissance locale des panneaux, il est nécessaire d'apporter quelques améliorations matérielles.

De la forme d'une clé USB, et fonctionnant directement sur un port USB 3.0, la Neural Compute Stick 2 d'Intel est un accélérateur d'intelligence artificielle autonome.

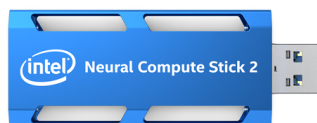


FIGURE 3.36 – Neural Compute Stick 2 par Intel

Au coeur du NCS 2 réside la dernière génération de l'unité de traitement de la vision (VPU) d'Intel, la VPU Movidius Myriad X, qui permet d'améliorer les performances jusqu'à 8 fois par rapport au premier NCS, malgré sa taille compacte.

Aujourd'hui la Neural Compute Stick 2 est quand même une merveille dans l'âge moderne des ordinateurs. Intégrer un moteur de traitement neuronal dédié dans un dongle de la taille d'une clé USB 3.0 peut, lorsqu'elle est branchée sur un ordinateur, considérablement augmenter les performances d'apprentissage en profondeur par rapport au traitement traditionnel du processeur et du processeur graphique : Une amélioration très signifiante au niveau du rendement pour un ordinateur comme Raspberry Pi.

Spécifications techniques

- Dimensions : 72,5 x 27 x 14 mm
- Processeur de vision Intel Movidius Myriad X Vision Processing Unit (VPU)
- Connectivité : USB 3.0 type A
- Température de fonctionnement : 0 à 40°C
- OS pris en charge : Ubuntu 16.04.3 LTS (64 bit) ; CentOS* 7.4 (64 bit) ; Windows 10 (64 bit) ; macOS 10.14.4 ; Raspbian

3.5.2 Problèmes au niveau matériel

Quant aux composants matériels utilisés pour élaborer ce projet (Capteurs, ordinateurs...), les problèmes au niveau du câblage ne présentaient aucun obstacle en raison de la grande mobilité des composants utilisés : Raspberry Pi 4B étant un nano-ordinateur très puissant et mobile permettant une grande flexibilité de déplacement. La Picamera étant un module minuscule mais robuste et performant. Cependant le module GPS Neo 6M v2 présentait plusieurs problèmes techniques à savoir : L'incapacité d'acquérir une position GPS pendant le premier démarrage et la fragilité du module ainsi que de l'antenne résultant facilement en un module transmettant des informations erronées ou illisibles.

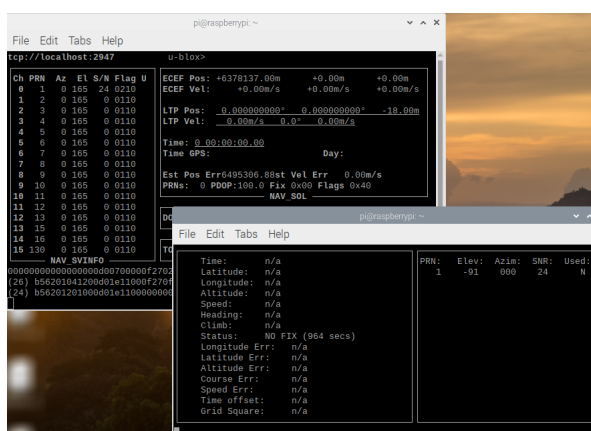


FIGURE 3.37 – Module GPS grillé

3.5.3 Influence de l'environnement du système

Il faut également souligner que l'environnement d'exécution du système pourrait également influencer la performance et le rendement. Concernant la détection des panneaux, nous attirons votre attention sur le fait que la qualité des panneaux est un facteur influençant les résultats du filtrage. Pendant notre expérimentation avec le système nous avons remarqué que les panneaux de circulations dans les villes du Maroc ne sont pas tous dans une condition reflétant un contrôle et une maintenance continue. Ceci résultant à des panneaux largement déformés ou avec des couleurs pales qui ne peuvent être détectés par le système. En général le module Picamera n'est pas fonctionnel pendant la nuit, et pour la reconnaissance faciale une bonne luminosité de l'environnement est toujours recommandé, tant sur la phase d'enregistrement des nouveaux visages que sur la phase de détection et reconnaissance. Pour remédier à cela, un autre modèle du Picamera est disponible sur le marché, Raspberry Pi Night Vision Camera, mais l'utilisation de ce modèle nécessite une réforme au niveau des mécanismes utilisés actuellement pour la détection et la reconnaissance. Ceci résulte donc dans un nouveau horizon que le projet pourrait atteindre dans les phases de développement à venir potentiellement.

4 Conclusion

Les résultats de ce projet étant satisfaisants, nous devons d'abord exprimer que cette expérience de projet nous a permis d'approfondir nos connaissances en guise d'apprentissage automatique, et de programmation embarquée, mais également en termes d'utilisation des bibliothèques open-source puissantes et flexibles, permettant de développer des systèmes puissants, indépendamment de la plateforme. Raspberry Pi fut un ordinateur puissant et capable de performer un grand nombre de calculs, même avec sa taille de "carte bancaire". Grâce à ce projet nous avons essayé de prendre le Raspberry Pi vers ces limites pour affirmer sa vraie puissance, et dévoiler de nouveaux horizons, pour les projets à grande mobilité, et haute performance, réalisables grâce à ce nano-ordinateur. Nous avons également enrichi notre bagage d'outils en termes de techniques de traitement d'image par le biais des outils de filtrage, et apprentissage automatique.

Bibliographie

- [1] J. HURWITZ et D. KIRSCH, *Machine Learning*. adresse : <https://ibm.co/3jhME2i>.
- [2] G. HALFACREE, *Raspberry Pi Beginner's Guide 4th Edition*. adresse : <https://bit.ly/3gW6Jd8>.
- [3] B. L, *Python : tout savoir sur le principal langage Big Data et Machine Learning*. adresse : <https://bit.ly/3y1Sp8n>.
- [4] T. CHABOUD, *OpenCV, c'est quoi ?* Adresse : <https://bit.ly/3h86F8Z>.
- [5] B. L, *TensorFlow : tout savoir sur la bibliothèque Machine Learning open source*. adresse : <https://bit.ly/3qrmDiM>.
- [6] *Overpass API*. adresse : https://wiki.openstreetmap.org/wiki/Overpass_API.